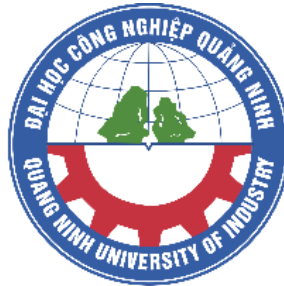


**BỘ CÔNG THƯƠNG
TRƯỜNG ĐẠI HỌC CÔNG NGHIỆP QUẢNG NINH**



ĐỒ ÁN TỐT NGHIỆP ĐẠI HỌC

ĐỀ TÀI :

**XÂY DỰNG ỨNG DỤNG WEB QUẢN LÝ CỬA HÀNG THỨC ĂN
NHANH**

Sinh viên:

Soulita PHANTHAVONG

Lớp:

Công nghệ phần mềm K15

Ngành học:

Công nghệ Thông tin

Khóa học:

2022 - 2026

Hình thức đào tạo:

Chính quy

Giảng viên hướng dẫn:

Nguyễn Phương Thảo

Quảng Ninh, tháng 06 năm 2026

BỘ CÔNG THƯƠNG
TRƯỜNG ĐẠI HỌC CÔNG NGHIỆP QUẢNG NINH



ĐỒ ÁN TỐT NGHIỆP ĐẠI HỌC
ĐỀ TÀI :
XÂY DỰNG ỨNG DỤNG WEB QUẢN LÝ CỬA HÀNG THỨC ĂN
NHANH

Sinh viên:	Soulita PHANTHAVONG
Lớp:	Công nghệ phần mềm K15
Ngành học:	Công nghệ Thông tin
Khóa học:	2022 - 2026
Hình thức đào tạo:	Chính quy
Giảng viên hướng dẫn:	Nguyễn Phương Thảo

Quảng Ninh, tháng 06 năm 2026

LỜI CẢM ƠN

Em xin chân thành cảm ơn Ban Giám hiệu Trường Đại học Công nghiệp Quảng Ninh, cùng toàn thể quý thầy cô Khoa Công nghệ Thông tin đã tận tình giảng dạy, truyền đạt cho em những kiến thức quý báu trong suốt quá trình học tập tại trường. Đây là nền tảng quan trọng giúp em có thể thực hiện và hoàn thành đồ án tốt nghiệp này.

Đặc biệt, em xin gửi lời cảm ơn sâu sắc tới cô Nguyễn Phương Thảo – giảng viên hướng dẫn, người đã trực tiếp chỉ bảo, định hướng và hỗ trợ em trong suốt quá trình thực hiện đề tài. Những góp ý chuyên môn và sự tận tâm của cô đã giúp em hoàn thiện sản phẩm cũng như báo cáo một cách tốt nhất.

Em cũng xin cảm ơn gia đình, bạn bè đã luôn bên cạnh, động viên và tạo điều kiện thuận lợi để em có thể tập trung học tập và hoàn thành đồ án đúng thời hạn.

Mặc dù đã cố gắng hết sức, nhưng do kiến thức và kinh nghiệm còn hạn chế nên báo cáo không tránh khỏi những thiếu sót. Em rất mong nhận được sự góp ý, nhận xét từ quý thầy cô để em có thể hoàn thiện hơn trong tương lai.

Quảng Ninh, ngày 20 tháng 06 năm 2026

Sinh viên thực hiện

Soulita PHANTHAVONG

NHẬN XÉT CỦA GIÁO VIÊN HƯỚNG DẪN

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Ngày tháng năm

GIÁO VIÊN HƯỚNG DẪN

(Ký và ghi rõ họ tên)

Mục Lục

CHƯƠNG 1: GIỚI THIỆU ĐỀ TÀI	1
1.1 Thực trạng.....	1
1.2 Lý do chọn đề tài	1
1.3 Mục tiêu của đề tài.....	1
1.4 Đối tượng sử dụng	2
1.5 Phạm vi nghiên cứu	2
1.6 Bố cục báo cáo.....	3
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG.....	4
2.1 Tổng quan về cơ sở lý thuyết.....	4
2.1.1 Mô hình Client – Server	4
2.1.2 Mô hình MVC (Model – View – Controller)	4
2.1.3 Kiến trúc RESTful API.....	4
2.2 Công nghệ sử dụng	5
2.2.1 Công nghệ phía Frontend	5
2.2.2 Công nghệ phía Backend.....	5
2.2.3 Hệ quản trị cơ sở dữ liệu	5
2.2.4 Các công nghệ hỗ trợ khác	6
2.3 Lý do lựa chọn công nghệ	6
2.4 Tổng kết chương.....	6
CHƯƠNG 3: PHÂN TÍCH HỆ THỐNG	7
3.1 Biểu đồ Use Case và các tác nhân (Actors).....	7
3.1.1 Tác nhân Khách hàng (Customer)	7
3.1.2 Tác nhân Nhân viên (Staff).....	7
3.1.3 Tác nhân Quản trị viên (Admin).....	8
3.1.4 Môi quan hệ giữa các tác nhân và hệ thống	8
3.1.5 Tổng kết.....	9
3.2 Yêu cầu chức năng (Functional Requirements).....	9
3.2.1 Chức năng tạo đơn hàng (Order Management)	9
3.2.2 Chức năng thanh toán (Payment Processing)	10
3.2.3 Chức năng quản lý bàn (Table Management).....	10
3.2.4 Chức năng quản lý thực đơn (Menu Management).....	11
3.2.5 Tổng kết.....	11

3.3 Yêu cầu phi chức năng (Non-Functional Requirements)	11
3.3.1 Bảo mật (Security – JWT)	11
3.3.2 Hiệu năng và tốc độ (Performance)	12
3.3.3 Tính dễ sử dụng (Usability)	12
3.3.4 Tổng kết	13
3.4 Các sơ đồ hệ thống (System Diagrams)	13
3.4.1 Biểu đồ Use Case (Use Case Diagram)	13
3.4.2 Biểu đồ hoạt động (Activity Diagram)	15
3.4.3 Biểu đồ lớp (Class Diagram)	16
3.4.4 Biểu đồ ERD (Entity Relationship Diagram)	17
3.4.5 Biểu đồ phân rã chức năng (Functional Decomposition Diagram)	18
3.4.6 Tổng kết	19
CHƯƠNG 4: THIẾT KẾ HỆ THỐNG	20
4.1 Kiến trúc hệ thống	20
4.1.1 Mô hình Client – Server	20
4.1.2 Kiến trúc phân tầng Backend	20
4.1.3 Kiến trúc RESTful API	21
4.1.4 Luồng hoạt động của hệ thống	21
4.1.5 Kết nối cơ sở dữ liệu	21
4.1.6 Đánh giá kiến trúc	22
4.2 Thiết kế cơ sở dữ liệu (Database Design)	22
4.2.1 Collection User	22
4.2.2 Collection Product	22
4.2.3 Collection Category	23
4.2.4 Collection Table	23
4.2.5 Collection Order	24
4.2.6 Embedded Document: OrderItem	24
4.2.7 Collection Payment	25
4.2.8 Nhận xét thiết kế	25
4.3 Thiết kế API (API Design)	25
4.3.1 API xác thực (Authentication API)	25
4.3.2 API quản lý người dùng (User API)	26
4.3.3 API quản lý sản phẩm (Product API)	26

4.3.4 API quản lý danh mục (Category API).....	27
4.3.5 API quản lý bàn (Table API).....	27
4.3.6 API quản lý đơn hàng (Order API).....	28
4.3.7 API thanh toán (Payment API)	28
4.3.8 API thống kê (Analytics API).....	28
4.3.9 Bảo mật API	29
4.4 Thiết kế giao diện người dùng (UI Design).....	29
CHƯƠNG 5: CÀI ĐẶT VÀ TRIỂN KHAI	36
5.1 Môi trường và công nghệ sử dụng.....	36
5.1.1 Môi trường phát triển.....	36
5.1.2 Công nghệ sử dụng	36
5.2 Cài đặt hệ thống.....	36
5.2.1 Chuẩn bị môi trường.....	36
5.2.2 Cài đặt Backend.....	36
5.2.3 Cài đặt Frontend	37
5.3 Cấu hình hệ thống.....	37
5.4 Dữ liệu mẫu (Seed Data)	37
5.4.1 Nội dung dữ liệu mẫu	37
5.4.2 Cách thực hiện	37
5.5 Kiểm thử sau cài đặt	38
5.6 Các đoạn mã nguồn tiêu biểu	38
5.6.1 Xác thực và phân quyền người dùng.....	38
5.6.2 Xử lý tạo đơn hàng	39
5.6.3 Quản lý bàn và mã QR	41
5.6.4 Quản lý thực đơn	42
5.6.5 Thanh toán đơn hàng	43
5.6.6 Thống kê doanh thu	44
5.7 Tổng kết chương.....	47
CHƯƠNG 6: KIỂM THỬ HỆ THỐNG	48
6.1 Kiểm thử chức năng (Functional Testing)	48
6.1.1 Bảng test case chi tiết	48
6.1.2 Phân tích kiểm thử theo từng module.....	49
6.1.3 Kết quả kiểm thử	52

6.1.4 Nhận xét.....	53
6.2 Kiểm thử khả dụng (Usability Testing)	53
6.2.1 Phương pháp kiểm thử.....	53
6.2.2 Kịch bản kiểm thử thực tế	54
6.2.3 Các tiêu chí đánh giá	57
6.2.4 Các vấn đề phát hiện và cải tiến	57
6.2.5 Kết quả kiểm thử khả dụng.....	58
6.2.6 Nhận xét.....	58
6.3 Theo dõi lỗi và khắc phục (Bug Tracking)	58
6.3.1 Phương pháp quản lý lỗi.....	58
6.3.2 Phân loại mức độ lỗi.....	59
6.3.3 Quy trình xử lý lỗi	59
6.3.4 Các lỗi tiêu biểu đã xử lý.....	61
6.3.5 Kết quả đạt được.....	62
6.4 Kiểm thử hiệu năng (Performance Testing).....	62
6.4.1 Load Testing	63
6.4.2 Stress Testing	64
6.4.3 Endurance Testing	64
6.4.4 Đánh giá tổng thể hiệu năng	65
6.4.5 Hướng tối ưu (định hướng phát triển)	65
6.4.6 Kết luận	66
6.5 Tổng kết chương.....	66
KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	67
7.1 Kết quả đạt được.....	67
7.2 Ưu điểm của hệ thống.....	67
7.3 Nhược điểm	67
7.4 Hướng phát triển trong tương lai.....	67
7.5 Tổng kết.....	68
Tài Liệu tham khảo.....	69

Danh mục các ký hiệu, chữ viết tắt

STT	Ký hiệu	Tên đầy đủ	Diễn giải
1	API	Application Programming Interface	Giao diện lập trình ứng dụng cho phép các hệ thống giao tiếp với nhau
2	Auth	Authentication	Quá trình xác thực người dùng
3	Authorization	Authorization	Phân quyền truy cập hệ thống
4	DB	Database	Cơ sở dữ liệu
5	DELETE	DELETE	Phương thức xóa dữ liệu
6	DOM	Document Model	Object Mô hình biểu diễn cấu trúc tài liệu HTML
7	Endurance Testing	Endurance Testing	Kiểm thử khả năng hoạt động lâu dài
8	Functional Testing	Functional Testing	Kiểm thử chức năng của hệ thống
9	GET	GET	Phương thức lấy dữ liệu từ server
10	HTTP	HyperText Transfer Protocol	Giao thức truyền tải dữ liệu trên web
11	JSON	JavaScript Notation	Object Định dạng dữ liệu dùng để trao đổi giữa client và server
12	JWT	JSON Web Token	Chuẩn xác thực và phân quyền người dùng
13	Load Testing	Load Testing	Kiểm thử khả năng chịu tải
14	Menu	Menu	Danh sách món ăn

STT	Ký hiệu	Tên đầy đủ	Diễn giải
15	MVC	Model – View – Controller	Mô hình thiết kế phần mềm chia hệ thống thành 3 thành phần: dữ liệu, giao diện và điều khiển
16	NoSQL	Not Only SQL	Hệ quản trị cơ sở dữ liệu phi quan hệ (ví dụ: MongoDB)
17	ODM	Object Data Modeling	Công cụ ánh xạ dữ liệu giữa đối tượng và cơ sở dữ liệu (ví dụ: Mongoose)
18	Order	Order	Đơn hàng trong hệ thống
19	PATCH	PATCH	Phương thức cập nhật một phần dữ liệu
20	Payment	Payment	Thanh toán đơn hàng
21	Performance Testing	Performance Testing	Kiểm thử hiệu năng hệ thống
22	POS	Point of Sale	Hệ thống bán hàng tại quầy
23	POST	POST	Phương thức tạo dữ liệu mới
24	PUT	PUT	Phương thức cập nhật dữ liệu
25	QR	Quick Response Code	Mã QR dùng để truy cập hệ thống và gọi món
26	REST	Representational State Transfer	Kiến trúc xây dựng dịch vụ web dựa trên giao thức HTTP
27	RESTful API	Representational State Transfer API	API tuân theo chuẩn REST, sử dụng các phương thức HTTP
28	SPA	Single Page Application	Ứng dụng web một trang, không cần tải lại toàn bộ trang
29	Stress Testing	Stress Testing	Kiểm thử trong điều kiện quá tải

STT	Ký hiệu	Tên đầy đủ	Diễn giải
30	Table	Table	Bàn trong cửa hàng
31	Test Case	Test Case	Kịch bản kiểm thử
32	UI	User Interface	Giao diện người dùng
33	Usability Testing	Usability Testing	Kiểm thử khả dụng (trải nghiệm người dùng)

Danh sách hình ảnh

Hình 3.1 Biểu đồ use case hệ thống Quản lý quán ăn	14
Hình 3.2 Biểu đồ activity diagram tạo đơn hàng.....	15
Hình 3.3 Biểu đồ Lớp (class diagram)	16
Hình 3.4 Biểu đồ ERD hệ thống Quản lý cửa hàng đồ ăn nhanh.....	18
Hình 3.5 Biểu đồ phân rã hệ thống Quản lý cửa hàng đồ ăn nhanh.....	19
Hình 4.1 Giao diện đăng nhập.....	29
Hình 4.2 Giao diện đăng ký.....	30
Hình 4.3 Giao diện chọn bàn (theo luồng không QR).....	30
Hình 4.4 giao diện gọi món	30
Hình 4.5 Giao diện theo dõi đơn hàng.....	31
Hình 4.6 Giao diện Profile và lịch sử	31
Hình 4.7 Giao diện DashBoard	31
Hình 4.8 Giao diện quản lý bàn.....	31
Hình 4.9 Giao diện quản lý Menu	32
Hình 4.10 Giao diện quản lý đơn hàng.....	32
Hình 4.11 Giao diện quản lý khách hàng	32
Hình 4.12 Giao diện quản lý nhân viên.....	33
Hình 4.13 Giao diện quản lý thanh toán.....	33
Hình 4.14 Giao diện quản lý hóa đơn.....	33
Hình 4.15 Giao diện quản lý POS	334
Hình 4.16 Giao diện quản lý đơn hàng staff.....	34
Hình 4.17 Giao diện thống kê Dashboard Staff	34
Hình 4.18 Giao diện cá nhân Staff	35
Hình 5.1 Đoạn code đăng ký tài khoản tạo JWT.....	38
Hình 5.2 Đoạn code xác thực đăng nhập.....	39
Hình 5.3 Đoạn code kiểm tra dữ liệu đơn hàng.....	40
Hình 5.4 Đoạn code kiểm tra và cập nhật tồn kho	41
Hình 5.5 Đoạn code sinh mã qr bàn ăn	42
Hình 5.6 Đoạn code tra cứu mã QR	42
Hình 5.7 Đoạn code kiểm tra trạng thái món ăn	42

Hình 5.8 Đoạn code kiểm tra số lượng tồn kho.....	42
Hình 5.9 Đoạn code trạng thái thanh toán đơn hàng.....	43
Hình 5.10 Đoạn code tạo giao dịch thanh toán	43
Hình 5.11 Đoạn code cập nhật trạng thái đơn hàng sau thanh toán	44
Hình 5.12 Đoạn code thống kê doanh thu theo ngày	45
Hình 5.13 Đoạn code thống kê doanh thu theo tháng	46
Hình 5.14 Đoạn code toongr hợp dữ liệu	46

Danh sách các bảng

Bảng 6.1 Bảng Test case tiêu biểu.....	48
Bảng 6.2 Bảng đánh giá.....	57
Bảng 6.3 Bảng phân loại lỗi	59

MỞ ĐẦU

Trong bối cảnh công nghệ thông tin đang phát triển mạnh mẽ và đóng vai trò quan trọng trong hầu hết các lĩnh vực của đời sống, việc ứng dụng các giải pháp phần mềm vào hoạt động kinh doanh ngày càng trở nên cần thiết. Đặc biệt, trong lĩnh vực dịch vụ ăn uống – nơi yêu cầu tốc độ phục vụ nhanh, độ chính xác cao và khả năng quản lý hiệu quả – việc áp dụng công nghệ không chỉ giúp nâng cao chất lượng dịch vụ mà còn tối ưu hóa quy trình vận hành, giảm thiểu sai sót và chi phí nhân lực.

Thực tế cho thấy, nhiều cửa hàng thức ăn nhanh hiện nay vẫn đang sử dụng các phương pháp quản lý thủ công hoặc bán thủ công như ghi chép đơn hàng bằng giấy, trao đổi thông tin qua nhân viên phục vụ hoặc sử dụng các công cụ rời rạc. Điều này dễ dẫn đến tình trạng nhầm lẫn đơn hàng, chậm trễ trong xử lý, khó kiểm soát doanh thu và không đáp ứng được nhu cầu ngày càng cao của khách hàng trong thời đại số.

Xuất phát từ những vấn đề trên, đề tài “Xây dựng ứng dụng quản lý cửa hàng thức ăn nhanh” được thực hiện với mục tiêu xây dựng một hệ thống phần mềm hỗ trợ toàn diện cho hoạt động quản lý và vận hành của cửa hàng. Hệ thống hướng đến việc số hóa quy trình gọi món thông qua mã QR, hỗ trợ quản lý đơn hàng, bàn ăn, thực đơn, cũng như cung cấp các chức năng dành cho nhân viên và quản trị viên nhằm nâng cao hiệu quả làm việc.

Đề tài tập trung nghiên cứu và phát triển một ứng dụng web với kiến trúc hiện đại, sử dụng các công nghệ phổ biến trong phát triển phần mềm như React cho giao diện người dùng, Node.js và Express.js cho phía máy chủ, cùng với MongoDB để lưu trữ và quản lý dữ liệu. Việc lựa chọn các công nghệ này nhằm đảm bảo tính linh hoạt, khả năng mở rộng và phù hợp với yêu cầu thực tế của hệ thống.

Báo cáo đồ án được trình bày với các nội dung chính bao gồm: giới thiệu tổng quan về đề tài, cơ sở lý thuyết và công nghệ sử dụng, phân tích và thiết kế hệ thống, quá trình cài đặt – triển khai, kiểm thử hệ thống và cuối cùng là kết luận cùng hướng phát triển trong tương lai. Thông qua các nội dung này, báo cáo không chỉ mô tả quá trình xây dựng hệ thống mà còn thể hiện khả năng vận dụng kiến thức đã học vào giải quyết một bài toán thực tế.

Với mục tiêu xây dựng một sản phẩm có tính ứng dụng cao, đề tài hy vọng sẽ góp phần nhỏ vào việc nâng cao hiệu quả quản lý trong lĩnh vực kinh doanh thức ăn nhanh, đồng thời là cơ hội để em củng cố kiến thức, rèn luyện kỹ năng và tích lũy kinh nghiệm thực tiễn trước khi bước vào môi trường làm việc chuyên nghiệp.

Em xin chân thành cảm ơn

CHƯƠNG 1: GIỚI THIỆU ĐỀ TÀI

1.1 Thực trạng

Trong những năm gần đây, ngành dịch vụ ăn uống, đặc biệt là các cửa hàng thức ăn nhanh, đang phát triển mạnh mẽ nhằm đáp ứng nhu cầu ngày càng cao của khách hàng về tốc độ phục vụ và chất lượng dịch vụ. Tuy nhiên, thực tế cho thấy nhiều cửa hàng vẫn đang áp dụng các phương pháp quản lý truyền thống như ghi chép đơn hàng bằng giấy, trao đổi thông tin qua nhân viên phục vụ hoặc sử dụng các công cụ rời rạc, thiếu tính đồng bộ.

Các phương pháp này tồn tại nhiều hạn chế như dễ xảy ra sai sót trong quá trình ghi nhận đơn hàng, nhầm lẫn món ăn, chậm trễ trong việc truyền thông tin giữa các bộ phận, cũng như khó khăn trong việc theo dõi doanh thu và quản lý dữ liệu. Bên cạnh đó, việc phụ thuộc quá nhiều vào con người còn làm giảm hiệu suất làm việc và khó mở rộng quy mô khi lượng khách hàng tăng cao.

Trong bối cảnh công nghệ số đang ngày càng phổ biến, những hạn chế trên đặt ra yêu cầu cần có một giải pháp quản lý hiện đại, tự động hóa và đồng bộ hơn nhằm nâng cao hiệu quả vận hành và chất lượng phục vụ.

1.2 Lý do chọn đề tài

Xuất phát từ những vấn đề thực tiễn nêu trên, việc xây dựng một hệ thống quản lý cửa hàng thức ăn nhanh dựa trên nền tảng công nghệ thông tin là cần thiết và có ý nghĩa thực tiễn cao. Việc ứng dụng công nghệ không chỉ giúp tối ưu hóa quy trình vận hành mà còn góp phần nâng cao trải nghiệm của khách hàng.

Đặc biệt, xu hướng sử dụng mã QR trong việc gọi món và thanh toán đang ngày càng phổ biến nhờ tính tiện lợi, nhanh chóng và hạn chế tiếp xúc trực tiếp. Bên cạnh đó, việc tích hợp hệ thống POS (Point of Sale) giúp quản lý đơn hàng, doanh thu và hoạt động kinh doanh một cách hiệu quả hơn.

Vì vậy, đề tài “Xây dựng ứng dụng quản lý cửa hàng thức ăn nhanh” được lựa chọn nhằm giải quyết các vấn đề tồn tại trong mô hình quản lý truyền thống, đồng thời hướng đến việc xây dựng một hệ thống hiện đại, dễ sử dụng và có khả năng mở rộng trong tương lai.

1.3 Mục tiêu của đề tài

Mục tiêu chính của đề tài là xây dựng một ứng dụng web hỗ trợ quản lý toàn diện hoạt động của cửa hàng thức ăn nhanh, cụ thể như sau:

- Xây dựng hệ thống gọi món thông qua mã QR giúp khách hàng có thể tự đặt món một cách nhanh chóng và thuận tiện.

- Phát triển hệ thống quản lý đơn hàng (Order Management) cho phép theo dõi trạng thái đơn hàng theo thời gian thực.
- Xây dựng chức năng quản lý bàn (Table Management) giúp tối ưu hóa việc sắp xếp và phục vụ khách hàng.
- Thiết kế hệ thống POS hỗ trợ nhân viên trong việc xử lý thanh toán và kiểm soát doanh thu.
- Xây dựng hệ thống quản trị (Admin Panel) cho phép quản lý thực đơn, người dùng và các hoạt động của hệ thống.
- Đảm bảo hệ thống có giao diện thân thiện, dễ sử dụng và có khả năng mở rộng.

1.4 Đối tượng sử dụng

Hệ thống được thiết kế phục vụ cho ba nhóm đối tượng chính:

- **Khách hàng (Customer):** Sử dụng hệ thống để quét mã QR tại bàn, xem thực đơn và đặt món trực tiếp mà không cần thông qua nhân viên phục vụ. Điều này giúp tiết kiệm thời gian và nâng cao trải nghiệm người dùng.
- **Nhân viên (Staff):** Sử dụng hệ thống để theo dõi và xử lý đơn hàng, hỗ trợ khách hàng khi cần thiết, đồng thời thực hiện các thao tác thanh toán thông qua hệ thống POS.
- **Quản trị viên (Admin):** Quản lý toàn bộ hệ thống bao gồm thực đơn, bàn, đơn hàng, người dùng và theo dõi hoạt động kinh doanh. Đây là nhóm có quyền truy cập cao nhất và chịu trách nhiệm vận hành hệ thống.

1.5 Phạm vi nghiên cứu

Đề tài tập trung vào việc xây dựng một ứng dụng web phục vụ quản lý cửa hàng thức ăn nhanh với các chức năng cơ bản và cần thiết. Phạm vi cụ thể như sau:

- Xây dựng hệ thống dưới dạng web application, có thể truy cập thông qua trình duyệt trên các thiết bị như máy tính, máy tính bảng và điện thoại.
- Tập trung vào các chức năng chính như: gọi món qua QR, quản lý đơn hàng, quản lý bàn, quản lý thực đơn và thanh toán.
- Không phát triển ứng dụng di động (mobile native) trong phạm vi đề tài.
- Hệ thống chưa tích hợp các cổng thanh toán trực tuyến phức tạp mà tập trung vào mô hình thanh toán tại quầy.
- Dữ liệu được quản lý ở mức cơ bản, phục vụ cho mục đích học tập và mô phỏng thực tế.

1.6 Bố cục báo cáo

Báo cáo đồ án được tổ chức thành các chương chính nhằm trình bày đầy đủ quá trình nghiên cứu, phân tích, thiết kế và xây dựng hệ thống. Cụ thể như sau:

- **Chương 1: Giới thiệu đề tài**

Trình bày tổng quan về đề tài bao gồm thực trạng, lý do lựa chọn, mục tiêu, đối tượng sử dụng, phạm vi nghiên cứu và bố cục báo cáo.

- **Chương 2: Cơ sở lý thuyết và công nghệ sử dụng**

Giới thiệu các kiến thức nền tảng, mô hình hệ thống và các công nghệ được sử dụng trong quá trình phát triển.

- **Chương 3: Phân tích hệ thống**

Phân tích yêu cầu hệ thống, xây dựng các mô hình như Use Case, Activity Diagram, Class Diagram và ERD.

- **Chương 4: Thiết kế hệ thống**

Trình bày chi tiết kiến trúc hệ thống, thiết kế cơ sở dữ liệu, giao diện người dùng và các API.

- **Chương 5: Cài đặt và triển khai**

Mô tả quá trình xây dựng, cài đặt môi trường và triển khai hệ thống.

- **Chương 6: Kiểm thử hệ thống**

Kiểm thử các chức năng cũng như lỗi trước khi sử dụng

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG

2.1 Tổng quan về cơ sở lý thuyết

Để xây dựng một hệ thống phần mềm hoàn chỉnh, việc nắm vững các cơ sở lý thuyết là yếu tố quan trọng giúp đảm bảo tính đúng đắn, khả năng mở rộng và hiệu quả trong quá trình phát triển. Trong phạm vi đề tài, các mô hình kiến trúc phần mềm và phương pháp phát triển hệ thống được lựa chọn phù hợp với bài toán quản lý cửa hàng thức ăn nhanh.

2.1.1 Mô hình Client – Server

Mô hình Client – Server là một kiến trúc phổ biến trong phát triển ứng dụng web, trong đó hệ thống được chia thành hai thành phần chính:

- **Client (máy khách):** Là nơi người dùng tương tác trực tiếp với hệ thống thông qua giao diện, thường là trình duyệt web.
- **Server (máy chủ):** Là nơi xử lý logic nghiệp vụ, tiếp nhận yêu cầu từ client, xử lý và trả về kết quả.

Ưu điểm của mô hình này là dễ triển khai, dễ bảo trì và có khả năng mở rộng cao. Trong đề tài, kiến trúc Client – Server được áp dụng để tách biệt phần giao diện người dùng và phần xử lý dữ liệu, giúp hệ thống hoạt động linh hoạt và hiệu quả hơn.

2.1.2 Mô hình MVC (Model – View – Controller)

Mô hình MVC là một mô hình thiết kế phần mềm giúp tách biệt các thành phần của hệ thống thành ba phần riêng biệt:

- **Model:** Quản lý dữ liệu và logic nghiệp vụ.
- **View:** Hiển thị dữ liệu cho người dùng.
- **Controller:** Xử lý yêu cầu từ người dùng và điều phối giữa Model và View.

Việc áp dụng mô hình MVC giúp hệ thống dễ dàng phát triển, bảo trì và mở rộng. Trong đề tài, tư duy thiết kế theo hướng MVC được áp dụng ở cả frontend và backend nhằm đảm bảo cấu trúc mã nguồn rõ ràng và dễ quản lý.

2.1.3 Kiến trúc RESTful API

REST (Representational State Transfer) là một phong cách kiến trúc dùng để xây dựng các dịch vụ web thông qua giao thức HTTP. RESTful API sử dụng các phương thức HTTP như GET, POST, PUT, DELETE để thực hiện các thao tác trên tài nguyên.

Các đặc điểm chính của RESTful API bao gồm:

- Không trạng thái (Stateless)
- Tách biệt giữa client và server

- Sử dụng định dạng dữ liệu phổ biến như JSON

Trong đề tài, RESTful API được sử dụng để xây dựng hệ thống giao tiếp giữa frontend và backend, đảm bảo tính linh hoạt và dễ tích hợp.

2.2 Công nghệ sử dụng

Để hiện thực hóa hệ thống, đề tài sử dụng một số công nghệ phổ biến và phù hợp với xu hướng phát triển hiện nay.

2.2.1 Công nghệ phía Frontend

Frontend là phần giao diện người dùng, nơi khách hàng và nhân viên trực tiếp tương tác với hệ thống.

- React: Là một thư viện JavaScript được phát triển bởi Meta, dùng để xây dựng giao diện người dùng. React cho phép xây dựng UI theo dạng component, giúp tái sử dụng mã nguồn và tăng hiệu suất thông qua cơ chế Virtual DOM.

Ưu điểm của React:

- Hiệu suất cao
- Dễ tái sử dụng component
- Hỗ trợ phát triển ứng dụng một trang (SPA)

2.2.2 Công nghệ phía Backend

Backend là nơi xử lý logic nghiệp vụ và quản lý dữ liệu của hệ thống.

- Node.js: Là môi trường chạy JavaScript phía server, cho phép xây dựng các ứng dụng mạng nhanh và có khả năng xử lý nhiều kết nối đồng thời.
- Express.js: Là một framework nhẹ cho Node.js, hỗ trợ xây dựng API nhanh chóng và dễ dàng.

Ưu điểm:

- Xử lý bất đồng bộ hiệu quả
- Tốc độ cao
- Phù hợp với ứng dụng thời gian thực

2.2.3 Hệ quản trị cơ sở dữ liệu

- MongoDB: Là hệ quản trị cơ sở dữ liệu NoSQL, lưu trữ dữ liệu dưới dạng document (JSON).

Ưu điểm của MongoDB:

- Linh hoạt trong thiết kế dữ liệu

- Dễ mở rộng
- Phù hợp với các ứng dụng web hiện đại

2.2.4 Các công nghệ hỗ trợ khác

Ngoài các công nghệ chính, hệ thống còn sử dụng một số công cụ và thư viện hỗ trợ:

- JWT (JSON Web Token): Dùng để xác thực và phân quyền người dùng.
- Axios: Thư viện hỗ trợ gửi HTTP request từ frontend đến backend.
- Mongoose: Thư viện ODM giúp làm việc với MongoDB dễ dàng hơn.
- Git: Công cụ quản lý phiên bản mã nguồn.
- Postman: Công cụ kiểm thử API.

2.3 Lý do lựa chọn công nghệ

Việc lựa chọn công nghệ trong đề tài được cân nhắc dựa trên các tiêu chí như tính phổ biến, khả năng mở rộng, hiệu suất và mức độ phù hợp với bài toán thực tế.

- Sử dụng JavaScript xuyên suốt (Frontend và Backend) giúp đồng nhất ngôn ngữ, giảm độ phức tạp trong quá trình phát triển.
- React giúp xây dựng giao diện nhanh, dễ bảo trì và nâng cao trải nghiệm người dùng.
- Node.js và Express.js cho phép xây dựng hệ thống backend linh hoạt, xử lý nhanh và phù hợp với các ứng dụng thời gian thực như quản lý đơn hàng.
- MongoDB phù hợp với dữ liệu linh hoạt, dễ thay đổi theo yêu cầu thực tế của hệ thống.
- RESTful API giúp hệ thống dễ dàng mở rộng và tích hợp với các dịch vụ khác trong tương lai.

2.4 Tổng kết chương

Trong chương này, đề tài đã trình bày các cơ sở lý thuyết và công nghệ được sử dụng trong quá trình xây dựng hệ thống. Việc lựa chọn mô hình Client – Server, kiến trúc RESTful API cùng với các công nghệ hiện đại như React, Node.js, Express.js và MongoDB đã tạo nền tảng vững chắc cho việc phát triển hệ thống quản lý cửa hàng thức ăn nhanh.

Những nội dung trình bày trong chương này sẽ là cơ sở để thực hiện việc phân tích và thiết kế hệ thống trong các chương tiếp theo.

CHƯƠNG 3: PHÂN TÍCH HỆ THỐNG

3.1 Biểu đồ Use Case và các tác nhân (Actors)

Trong quá trình phân tích hệ thống, biểu đồ Use Case được sử dụng để mô tả các chức năng chính của hệ thống và cách các tác nhân (actors) tương tác với hệ thống. Đây là công cụ quan trọng giúp xác định rõ phạm vi chức năng, từ đó làm cơ sở cho việc thiết kế và phát triển hệ thống.

Dựa trên yêu cầu thực tế của đề tài “Xây dựng ứng dụng quản lý cửa hàng thức ăn nhanh”, hệ thống được xác định gồm ba tác nhân chính: **Khách hàng (Customer)**, **Nhân viên (Staff)** và **Quản trị viên (Admin)**. Mỗi tác nhân có vai trò và quyền hạn khác nhau trong hệ thống.

3.1.1 Tác nhân Khách hàng (Customer)

Khách hàng là người trực tiếp sử dụng hệ thống để thực hiện việc gọi món thông qua mã QR tại bàn. Đây là tác nhân không cần đăng nhập nhưng vẫn có thể sử dụng các chức năng cơ bản của hệ thống.

Các chức năng chính của Customer:

- Quét mã QR để truy cập vào hệ thống theo bàn tương ứng
- Truy cập vào nền tảng để chọn bàn
- Xem thực đơn (menu) của cửa hàng
- Lựa chọn món ăn và thêm vào giỏ hàng
- Tạo đơn hàng (Order)
- Theo dõi trạng thái đơn hàng (đang xử lý, đã hoàn thành, đã thanh toán)
- Gửi ghi chú cho đơn hàng (nếu có)

Đặc điểm:

- Yêu cầu tài khoản đăng nhập
- Tương tác trực tiếp với giao diện web trên thiết bị cá nhân
- Dữ liệu gắn liền với mã bàn (table)

3.1.2 Tác nhân Nhân viên (Staff)

Nhân viên là người vận hành hệ thống tại cửa hàng, có nhiệm vụ theo dõi và xử lý các đơn hàng từ khách hàng, đồng thời thực hiện thanh toán.

Các chức năng chính của Staff:

- Đăng nhập hệ thống

- Xem danh sách đơn hàng theo thời gian thực
- Cập nhật trạng thái đơn hàng (đang xử lý, đã phục vụ, hoàn thành)
- Xác nhận và thực hiện thanh toán đơn hàng
- Hỗ trợ khách hàng trong quá trình sử dụng hệ thống

Đặc điểm:

- Có tài khoản đăng nhập với quyền hạn hạn chế
- Tương tác với hệ thống thông qua giao diện quản lý (POS và dashboard)
- Đóng vai trò trung gian giữa khách hàng và hệ thống

3.1.3 Tác nhân Quản trị viên (Admin)

Quản trị viên là người có quyền cao nhất trong hệ thống, chịu trách nhiệm quản lý toàn bộ hoạt động của ứng dụng.

Các chức năng chính của Admin:

- Đăng nhập hệ thống với quyền quản trị
- Quản lý thực đơn (thêm, sửa, xóa món ăn)
- Quản lý danh sách bàn (tạo, cập nhật, xóa bàn)
- Quản lý người dùng (tài khoản nhân viên)
- Theo dõi và quản lý toàn bộ đơn hàng
- Quản lý bàn (trạng thái bàn: trống, đang sử dụng)
- Xem báo cáo doanh thu

Đặc điểm:

- Có toàn quyền truy cập vào hệ thống
- Thực hiện các thao tác quản lý và cấu hình
- Đảm bảo hệ thống hoạt động ổn định và hiệu quả

3.1.4 Mối quan hệ giữa các tác nhân và hệ thống

Ba tác nhân trong hệ thống có mối quan hệ chặt chẽ và tương tác với nhau thông qua các chức năng của hệ thống:

- Khách hàng tạo đơn hàng thông qua giao diện QR hoặc trực tiếp trên nền tảng
- Nhân viên tiếp nhận và xử lý đơn hàng từ khách hàng

- Quản trị viên quản lý toàn bộ dữ liệu và hoạt động của hệ thống

Sự phân chia rõ ràng giữa các tác nhân giúp hệ thống đảm bảo tính bảo mật, phân quyền hợp lý và dễ dàng mở rộng trong tương lai.

3.1.5 Tổng kết

Thông qua việc xác định các tác nhân chính và các chức năng tương ứng, hệ thống đã được phân tích một cách rõ ràng về mặt nghiệp vụ. Đây là cơ sở quan trọng để xây dựng biểu đồ Use Case tổng thể và triển khai các chức năng cụ thể trong các phần tiếp theo của báo cáo.

3.2 Yêu cầu chức năng (Functional Requirements)

Yêu cầu chức năng mô tả các chức năng chính mà hệ thống cần cung cấp để đáp ứng nhu cầu của người sử dụng. Dựa trên phân tích nghiệp vụ của hệ thống quản lý cửa hàng thức ăn nhanh, các chức năng cốt lõi bao gồm: tạo đơn hàng, thanh toán, quản lý bàn và quản lý thực đơn.

3.2.1 Chức năng tạo đơn hàng (Order Management)

Đây là chức năng trung tâm của hệ thống, cho phép khách hàng thực hiện việc gọi món thông qua mã QR và gửi yêu cầu đến hệ thống.

Mô tả:

Khách hàng sử dụng thiết bị cá nhân quét mã QR tại bàn để truy cập vào giao diện thực đơn. Tại đây, khách hàng có thể lựa chọn món ăn, số lượng và gửi đơn hàng trực tiếp đến hệ thống mà không cần thông qua nhân viên.

Các chức năng chi tiết:

- Hiển thị danh sách món ăn theo menu
- Thêm món ăn vào giỏ hàng
- Cập nhật số lượng hoặc xóa món trong giỏ
- Gửi đơn hàng (create order)
- Gắn đơn hàng với mã bàn (table)
- Gửi ghi chú cho đơn hàng

Kết quả:

- Đơn hàng được lưu vào hệ thống với trạng thái ban đầu (ví dụ: “pending”)
- Nhân viên có thể theo dõi và xử lý đơn hàng theo thời gian thực

3.2.2 Chức năng thanh toán (Payment Processing)

Chức năng thanh toán hỗ trợ nhân viên hoàn tất giao dịch với khách hàng sau khi đơn hàng đã được phục vụ.

Mô tả:

Sau khi khách hàng sử dụng dịch vụ, nhân viên sẽ truy cập hệ thống để kiểm tra đơn hàng và thực hiện thanh toán. Hệ thống hỗ trợ hình thức thanh toán trực tiếp tại quầy.

Các chức năng chi tiết:

- Hiển thị thông tin chi tiết đơn hàng
- Tính tổng tiền dựa trên các món đã chọn
- Xác nhận thanh toán
- Cập nhật trạng thái đơn hàng thành “paid”
- Lưu thông tin thanh toán

Kết quả:

- Đơn hàng được đánh dấu là đã thanh toán
- Dữ liệu được lưu trữ phục vụ cho việc thống kê và báo cáo

3.2.3 Chức năng quản lý bàn (Table Management)

Chức năng này giúp kiểm soát trạng thái và thông tin của các bàn trong cửa hàng.

Mô tả:

Hệ thống cho phép quản lý danh sách bàn, bao gồm việc tạo mới, cập nhật và theo dõi trạng thái sử dụng của từng bàn.

Các chức năng chi tiết:

- Tạo bàn mới (thêm bàn)
- Cập nhật thông tin bàn (mã bàn, trạng thái)
- Xóa bàn khỏi hệ thống
- Gán mã QR cho từng bàn
- Theo dõi trạng thái bàn (trống, đang sử dụng)

Kết quả:

- Dễ dàng quản lý số lượng và trạng thái bàn

- Hỗ trợ phân luồng khách hàng hiệu quả

3.2.4 Chức năng quản lý thực đơn (Menu Management)

Chức năng này cho phép quản trị viên quản lý các món ăn được hiển thị trong hệ thống.

Mô tả:

Admin có thể thêm mới, chỉnh sửa hoặc xóa các món ăn trong thực đơn. Các thay đổi sẽ được cập nhật ngay lập tức trên giao diện của khách hàng.

Các chức năng chi tiết:

- Thêm món ăn mới (tên, giá, hình ảnh, mô tả)
- Chỉnh sửa thông tin món ăn
- Xóa món ăn khỏi thực đơn
- Phân loại món ăn (nếu có: đồ uống, món chính, ăn nhanh,...)
- Hiển thị danh sách món ăn

Kết quả:

- Thực đơn luôn được cập nhật chính xác
- Đảm bảo tính đồng bộ giữa hệ thống quản lý và giao diện người dùng

3.2.5 Tổng kết

Các yêu cầu chức năng trên đã bao quát những nghiệp vụ cốt lõi của hệ thống quản lý cửa hàng thức ăn nhanh. Việc xác định rõ các chức năng giúp đảm bảo hệ thống đáp ứng đúng nhu cầu thực tế, đồng thời tạo nền tảng cho việc thiết kế và triển khai trong các giai đoạn tiếp theo.

3.3 Yêu cầu phi chức năng (Non-Functional Requirements)

Bên cạnh các yêu cầu chức năng, hệ thống cần đáp ứng các yêu cầu phi chức năng nhằm đảm bảo hiệu năng, tính bảo mật và trải nghiệm người dùng. Đây là những tiêu chí quan trọng để đánh giá chất lượng tổng thể của hệ thống.

3.3.1 Bảo mật (Security – JWT)

Hệ thống áp dụng cơ chế xác thực và phân quyền người dùng thông qua JSON Web Token (JWT). Đây là phương pháp phổ biến trong các ứng dụng web hiện đại.

Mô tả:

- Người dùng (Staff, Admin) đăng nhập vào hệ thống bằng tài khoản.
- Sau khi xác thực thành công, hệ thống cấp một token (JWT) cho người dùng.

- Token được gửi kèm trong các request tiếp theo để xác thực quyền truy cập.

Các yêu cầu cụ thể:

- Bảo vệ các API quan trọng bằng middleware xác thực
- Phân quyền rõ ràng giữa Admin và Staff
- Mã hóa mật khẩu người dùng (bcrypt)
- Ngăn chặn truy cập trái phép vào hệ thống

Kết quả:

- Đảm bảo an toàn dữ liệu
- Kiểm soát quyền truy cập hiệu quả

3.3.2 Hiệu năng và tốc độ (Performance)

Hệ thống cần đảm bảo khả năng xử lý nhanh và ổn định, đặc biệt trong môi trường có nhiều người dùng truy cập đồng thời như giờ cao điểm tại cửa hàng.

Yêu cầu cụ thể:

- Thời gian phản hồi API nhanh (dưới 2–3 giây)
- Xử lý đồng thời nhiều đơn hàng
- Tối ưu truy vấn cơ sở dữ liệu
- Hạn chế reload trang nhờ sử dụng SPA (Single Page Application)

Kết quả:

- Đảm bảo trải nghiệm mượt mà cho người dùng
- Hệ thống hoạt động ổn định trong điều kiện thực tế

3.3.3 Tính dễ sử dụng (Usability)

Hệ thống được thiết kế với giao diện thân thiện, dễ sử dụng cho tất cả các đối tượng người dùng, kể cả người không am hiểu công nghệ.

Yêu cầu cụ thể:

- Giao diện rõ ràng, trực quan
- Quy trình gọi món đơn giản (ít bước)
- Hỗ trợ tốt trên thiết bị di động
- Thao tác nhanh, dễ hiểu

Kết quả:

- Người dùng dễ dàng tiếp cận và sử dụng hệ thống
- Giảm thời gian đào tạo nhân viên

3.3.4 Tổng kết

Các yêu cầu phi chức năng đóng vai trò quan trọng trong việc đảm bảo hệ thống không chỉ hoạt động đúng mà còn hoạt động tốt, an toàn và hiệu quả trong môi trường thực tế.

3.4 Các sơ đồ hệ thống (System Diagrams)

Để mô hình hóa và trực quan hóa hệ thống, đề tài sử dụng một số loại sơ đồ phổ biến trong phân tích và thiết kế phần mềm. Các sơ đồ này giúp mô tả rõ ràng cấu trúc, luồng xử lý và mối quan hệ giữa các thành phần trong hệ thống.

3.4.1 Biểu đồ Use Case (Use Case Diagram)**Mô tả:**

Biểu đồ Use Case thể hiện mối quan hệ giữa các tác nhân (Customer, Staff, Admin) và các chức năng của hệ thống.

Nội dung chính:

- Customer: tạo đơn hàng, xem menu, theo dõi đơn
- Staff: xử lý đơn hàng, thanh toán, xem dashboard
- Admin: quản lý bàn, quản lý menu, quản lý đơn hàng, quản lý nhân viên và người dùng, quản lý hóa đơn

Ý nghĩa:

- Giúp xác định phạm vi chức năng của hệ thống
- Là cơ sở để xây dựng các sơ đồ chi tiết hơn



Hình 3.1 Biểu đồ Use Case hệ thống Quản lý quán ăn

3.4.2 Biểu đồ hoạt động (Activity Diagram)

Mô tả:

Biểu đồ Activity được sử dụng để mô tả luồng xử lý chi tiết của một chức năng cụ thể trong hệ thống. Trong đề tài này, biểu đồ hoạt động được xây dựng dựa trên logic xử lý thực tế trong các service, tiêu biểu là chức năng **tạo đơn hàng**.

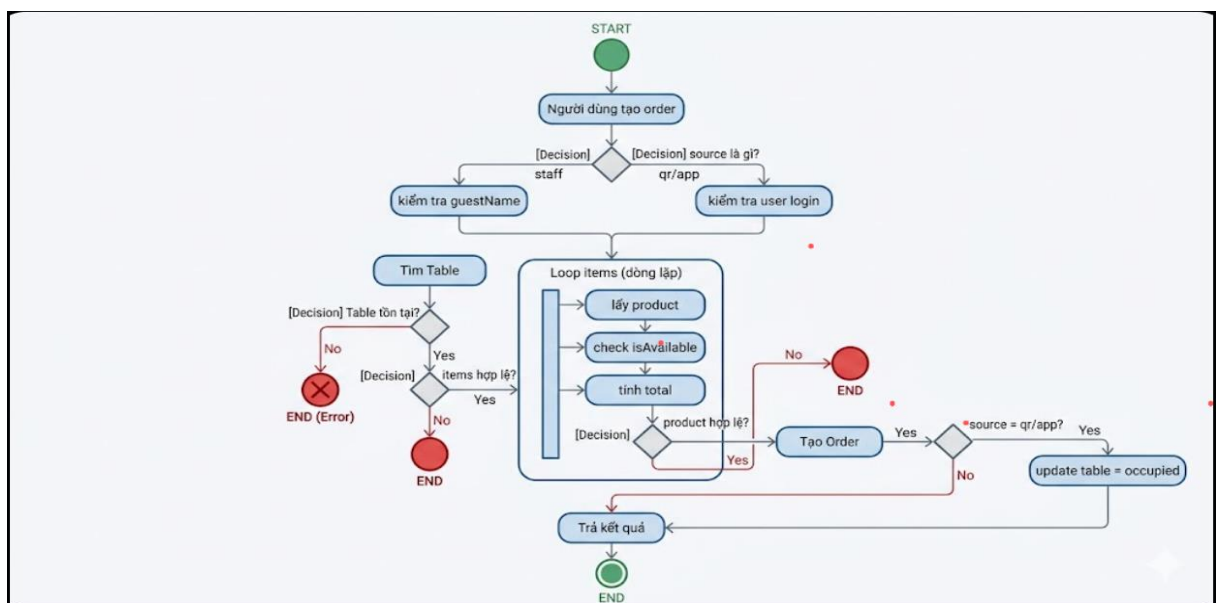
Biểu đồ thể hiện toàn bộ quá trình từ khi khách hàng hoặc nhân viên bắt đầu tạo đơn hàng cho đến khi hệ thống xử lý và lưu trữ dữ liệu thành công.

Luồng xử lý :

- Người dùng (Customer / Staff) bắt đầu tạo đơn hàng
- Xác định nguồn tạo đơn (staff / qr / app)
- Kiểm tra thông tin bàn
- Kiểm tra dữ liệu đầu vào (items, user, guestName)
- Lấy thông tin sản phẩm và tính tổng tiền
- Tạo đơn hàng trong hệ thống
- Cập nhật trạng thái bàn (nếu là QR/App)
- Trả kết quả về client

Ý nghĩa:

- Giúp mô tả rõ ràng cách hệ thống xử lý nghiệp vụ thực tế
- Thể hiện logic xử lý trong backend (service layer)
- Là cơ sở để kiểm thử và tối ưu hệ thống
- Giúp người đọc hiểu được luồng hoạt động từ đầu đến cuối



Hình 3.2 Biểu đồ activity diagram tạo đơn hàng

3.4.3 Biểu đồ lớp (Class Diagram)

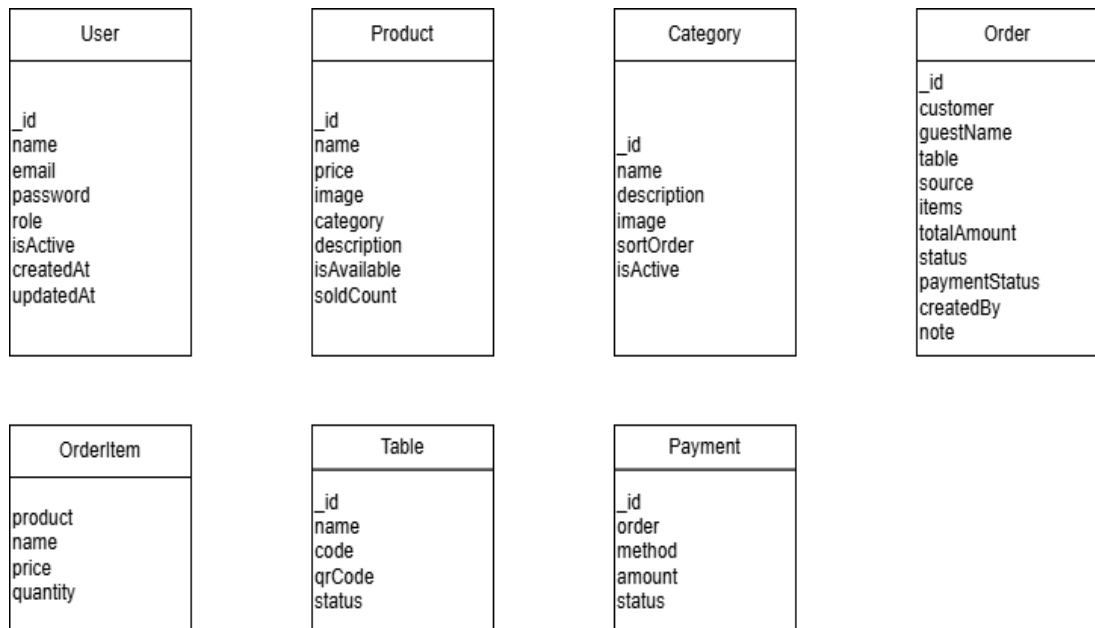
Mô tả:

Biểu đồ lớp (Class Diagram) thể hiện cấu trúc dữ liệu của hệ thống dưới dạng các lớp (class), bao gồm các thuộc tính và mối quan hệ giữa các lớp. Trong đề tài này, biểu đồ lớp được xây dựng dựa trực tiếp trên các model trong hệ thống sử dụng MongoDB và Mongoose.

Các lớp chính bao gồm: User, Product, Order, Table và Category. Ngoài ra, lớp OrderItem được sử dụng như một thành phần nhúng trong Order để biểu diễn chi tiết từng món ăn trong đơn hàng.

Các lớp chính:

- **User:** Lưu thông tin người dùng (admin, staff, customer)
- **Product:** Thông tin món ăn
- **Category:** Phân loại món ăn
- **Order:** Thông tin đơn hàng
- **OrderItem:** Chi tiết từng món trong đơn hàng
- **Table:** Thông tin bàn và QR code
- **Payment:** Thông tin thanh toán



Hình 3.3 Biểu đồ Lớp (class diagram)

Mối quan hệ giữa các lớp:

- Một Order thuộc một Table
- Một Order có nhiều OrderItem
- Một OrderItem tham chiếu đến một Product
- Một Product thuộc một Category
- Một Order có thể liên kết với User (customer, staff)
- Một Payment thuộc một Order

Ý nghĩa:

- Là cơ sở để thiết kế cơ sở dữ liệu
- Giúp mô hình hóa hệ thống theo hướng đối tượng
- Hỗ trợ lập trình và bảo trì hệ thống
- Đảm bảo tính nhất quán giữa thiết kế và triển khai

3.4.4 Biểu đồ ERD (Entity Relationship Diagram)**Mô tả:**

Biểu đồ ERD (Entity Relationship Diagram) được sử dụng để mô tả cấu trúc dữ liệu của hệ thống ở mức logic, thể hiện các thực thể (entity), thuộc tính và mối quan hệ giữa các thực thể trong cơ sở dữ liệu.

Trong đề tài này, ERD được xây dựng dựa trên các model trong hệ thống MongoDB, đảm bảo phản ánh chính xác cách dữ liệu được lưu trữ và liên kết với nhau.

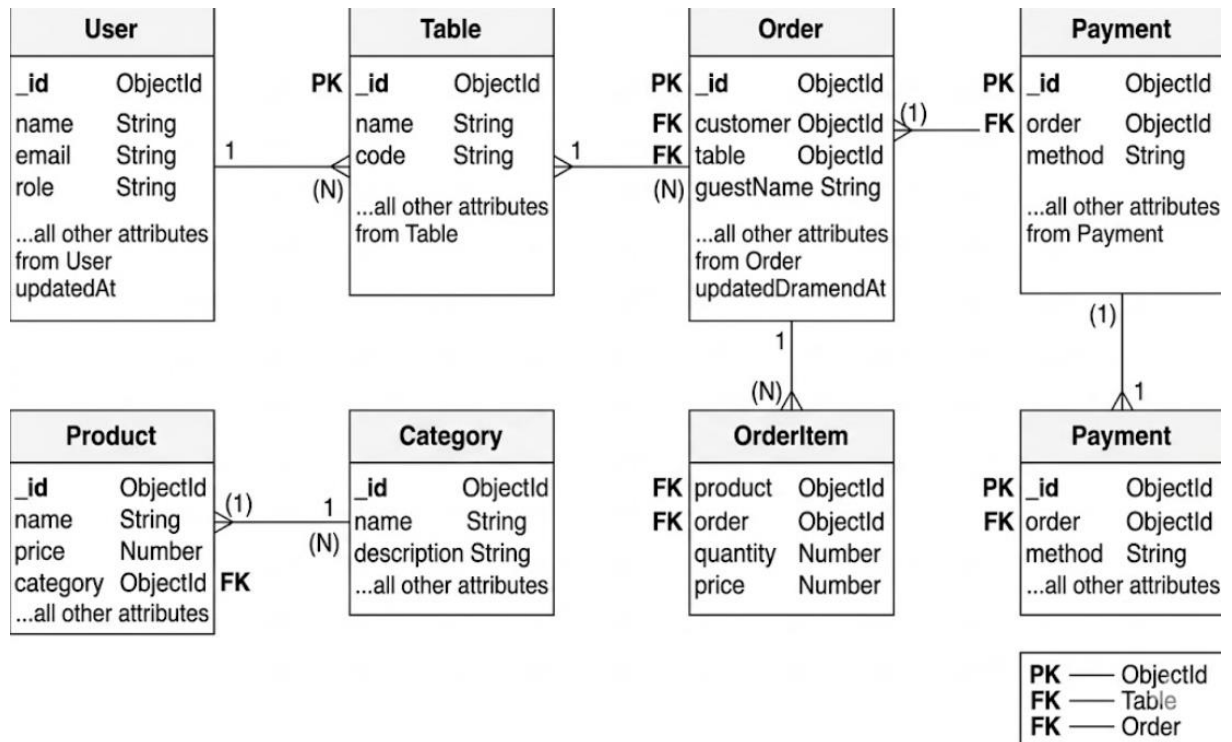
Các thực thể chính:

- **User:** Lưu thông tin người dùng (admin, staff, customer)
- **Order:** Lưu thông tin đơn hàng
- **Product:** Lưu thông tin món ăn
- **Table:** Lưu thông tin bàn
- **Category:** Phân loại món ăn
- **Payment:** Thông tin thanh toán
- **OrderItem:** Thực thể trung gian giữa Order và Product

Mối quan hệ:

- Một **Order** thuộc về một **Table** (N:1)
- Một **Order** có nhiều **OrderItem** (1)
- Một **OrderItem** liên kết với một **Product** (N:1)
- Một **Product** thuộc một **Category** (N:1)

- Một **Order** có thể thuộc về một **User** (customer hoặc staff)
- Một **Payment** thuộc về một **Order** (1:1)



Hình 3.4 Biểu đồ ERD hệ thống Quản lý cửa hàng đồ ăn nhanh

Ý nghĩa:

- Giúp thiết kế cơ sở dữ liệu logic một cách rõ ràng
- Thể hiện mối quan hệ giữa các bảng trong hệ thống
- Hỗ trợ xây dựng truy vấn và tối ưu dữ liệu
- Đảm bảo tính nhất quán và toàn vẹn dữ liệu

3.4.5 Biểu đồ phân rã chức năng (Functional Decomposition Diagram)

Biểu đồ phân rã chức năng được sử dụng để mô tả cấu trúc chức năng của hệ thống dưới dạng phân cấp. Từ chức năng tổng quát của hệ thống quản lý cửa hàng thức ăn nhanh, các chức năng được chia thành các nhóm nghiệp vụ nhỏ hơn nhằm giúp quá trình phân tích và thiết kế hệ thống trở nên rõ ràng hơn.

Các nhóm chức năng chính của hệ thống bao gồm:

- Quản lý người dùng.
- Quản lý thực đơn.
- Quản lý bàn.
- Quản lý đơn hàng.

- Quản lý thanh toán.
- Thống kê và báo cáo.

Mỗi nhóm chức năng tiếp tục được phân rã thành các chức năng con tương ứng để phục vụ cho từng nghiệp vụ cụ thể của cửa hàng.



Hình 3.5 Biểu đồ phân rã hệ thống Quản lý cửa hàng đồ ăn nhanh

3.4.6 Tổng kết

Các sơ đồ trong phần này giúp mô hình hóa toàn bộ hệ thống từ góc nhìn chức năng đến dữ liệu. Đây là bước quan trọng để chuyển từ giai đoạn phân tích sang thiết kế và triển khai hệ thống trong các chương tiếp theo.

CHƯƠNG 4: THIẾT KẾ HỆ THỐNG

4.1 Kiến trúc hệ thống

Trong đề tài “Xây dựng ứng dụng quản lý cửa hàng thức ăn nhanh”, hệ thống được thiết kế theo kiến trúc **Client – Server** kết hợp với mô hình **RESTful API**, nhằm đảm bảo tính linh hoạt, khả năng mở rộng và phù hợp với các ứng dụng web hiện đại.

4.1.1 Mô hình Client – Server

Hệ thống được chia thành hai thành phần chính:

- **Client (Frontend):**

Được xây dựng bằng thư viện ReactJS, cung cấp giao diện cho ba nhóm người dùng chính gồm khách hàng (Customer), nhân viên (Staff) và quản trị viên (Admin). Người dùng có thể thực hiện các thao tác như xem menu, đặt món qua QR, quản lý đơn hàng hoặc quản lý hệ thống thông qua trình duyệt web.

- **Server (Backend):**

Được xây dựng bằng Node.js và Express.js, chịu trách nhiệm xử lý toàn bộ logic nghiệp vụ của hệ thống, bao gồm:

- Xử lý yêu cầu từ client
- Xác thực và phân quyền người dùng (JWT)
- Thực hiện các thao tác với cơ sở dữ liệu MongoDB
- Trả kết quả về client dưới dạng JSON

4.1.2 Kiến trúc phân tầng Backend

Backend được tổ chức theo mô hình phân tầng nhằm đảm bảo tính rõ ràng và dễ bảo trì:

- **Controller Layer:** Tiếp nhận request từ client và chuyển tiếp đến service tương ứng
- **Service Layer:** Xử lý logic nghiệp vụ chính (tạo đơn hàng, thanh toán, quản lý bàn,...)
- **Model Layer:** Định nghĩa cấu trúc dữ liệu bằng Mongoose Schema và tương tác với MongoDB

Cách tổ chức này giúp tách biệt rõ ràng giữa xử lý dữ liệu và logic nghiệp vụ, giúp hệ thống dễ mở rộng và bảo trì.

4.1.3 Kiến trúc RESTful API

Hệ thống sử dụng RESTful API để giao tiếp giữa frontend và backend thông qua giao thức HTTP.

Đặc điểm chính:

- Sử dụng các phương thức HTTP: GET, POST, PUT, DELETE
- Dữ liệu trao đổi dưới dạng JSON
- Không lưu trạng thái (Stateless)
- Mỗi tài nguyên (resource) được định danh thông qua URL

Ví dụ một số API tiêu biểu:

- POST /api/orders → tạo đơn hàng
- GET /api/orders/ → lấy chi tiết đơn hàng
- PUT /api/orders//status → cập nhật trạng thái
- POST /api/payments → thanh toán đơn hàng

4.1.4 Luồng hoạt động của hệ thống

Quá trình xử lý một yêu cầu từ người dùng diễn ra theo các bước sau:

1. Người dùng thao tác trên giao diện (Client)
2. Client gửi HTTP Request đến API Server
3. Request được xử lý tại Controller
4. Controller gọi Service để xử lý logic nghiệp vụ
5. Service tương tác với Database thông qua Model
6. Kết quả được trả về Controller
7. Server trả Response về Client
8. Client cập nhật giao diện cho người dùng

4.1.5 Kết nối cơ sở dữ liệu

Hệ thống sử dụng MongoDB làm cơ sở dữ liệu NoSQL, kết nối thông qua thư viện Mongoose.

Dữ liệu được tổ chức theo dạng document (JSON-like), giúp linh hoạt trong việc lưu trữ và mở rộng. Các mối quan hệ giữa dữ liệu được thể hiện thông qua ObjectId và cơ chế populate.

4.1.6 Đánh giá kiến trúc

Ưu điểm:

- Tách biệt rõ ràng giữa frontend và backend
- Dễ dàng mở rộng và phát triển thêm tính năng
- Phù hợp với ứng dụng web hiện đại
- Dễ triển khai và bảo trì

Nhược điểm:

- Cần quản lý tốt API để tránh phức tạp khi hệ thống lớn
- Phụ thuộc vào kết nối mạng giữa client và server

4.2 Thiết kế cơ sở dữ liệu (Database Design)

Hệ thống sử dụng MongoDB làm cơ sở dữ liệu NoSQL để lưu trữ dữ liệu. Dữ liệu được tổ chức dưới dạng các collection tương ứng với các thực thể trong hệ thống. Mỗi collection được định nghĩa thông qua Mongoose Schema, giúp đảm bảo tính nhất quán và dễ dàng thao tác dữ liệu.

4.2.1 Collection User

Mô tả:

Lưu trữ thông tin người dùng trong hệ thống, bao gồm quản trị viên (Admin), nhân viên (Staff) và khách hàng (Customer).

Các trường chính:

- `_id`: ObjectId – Khóa chính
- `name`: String – Tên người dùng
- `email`: String – Email đăng nhập (duy nhất)
- `password`: String – Mật khẩu (được mã hóa bằng bcrypt)
- `role`: String – Vai trò (admin | staff | customer)
- `isActive`: Boolean – Trạng thái hoạt động
- `createdAt, updatedAt`: Date – Thời gian tạo và cập nhật

4.2.2 Collection Product

Mô tả:

Lưu trữ thông tin các món ăn trong hệ thống.

Các trường chính:

- `_id`: ObjectId
- `name`: String – Tên món ăn
- `price`: Number – Giá
- `image`: String – Hình ảnh
- `description`: String – Mô tả
- `category`: ObjectId (ref Category) – Thuộc danh mục
- `isAvailable`: Boolean – Trạng thái bán
- `soldCount`: Number – Số lượng đã bán
- `createdAt`, `updatedAt`: Date

4.2.3 Collection Category

Mô tả:

Phân loại các món ăn trong menu.

Các trường chính:

- `_id`: ObjectId
- `name`: String – Tên danh mục
- `description`: String – Mô tả
- `image`: String – Hình ảnh
- `sortOrder`: Number – Thứ tự hiển thị
- `isActive`: Boolean – Trạng thái
- `createdAt`, `updatedAt`: Date

4.2.4 Collection Table

Mô tả:

Quản lý thông tin bàn và QR code để khách đặt món.

Các trường chính:

- `_id`: ObjectId
- `name`: String – Tên bàn
- `code`: String – Mã bàn (duy nhất)
- `qrCode`: String – Link QR code

- status: String – Trạng thái (available | occupied)
- createdAt, updatedAt: Date

4.2.5 Collection Order

Mô tả:

Lưu trữ thông tin đơn hàng của khách hàng.

Các trường chính:

- _id: ObjectId
- customer: ObjectId (ref User) – Khách hàng (nếu có)
- guestName: String – Tên khách (nếu order bởi staff)
- table: ObjectId (ref Table) – Bàn đặt
- source: String – Nguồn tạo đơn (staff | qr | app)
- items: Array (OrderItem) – Danh sách món
- totalAmount: Number – Tổng tiền
- status: String – Trạng thái đơn
- paymentStatus: String – Trạng thái thanh toán
- createdBy: ObjectId (ref User) – Nhân viên tạo đơn
- note: String – Ghi chú
- createdAt, updatedAt: Date

4.2.6 Embedded Document: OrderItem

Mô tả:

Là document nhúng trong Order, dùng để lưu chi tiết từng sản phẩm trong đơn hàng.

Các trường:

- product: ObjectId (ref Product)
- name: String – Tên sản phẩm tại thời điểm đặt
- price: Number – Giá tại thời điểm đặt
- quantity: Number – Số lượng

Lưu ý:

Việc sử dụng embedded document giúp giảm số lần truy vấn và tăng hiệu năng khi lấy dữ liệu đơn hàng.

4.2.7 Collection Payment**Mô tả:**

Lưu thông tin thanh toán của đơn hàng.

Các trường chính:

- `_id`: ObjectId
- `order`: ObjectId (ref Order) – Đơn hàng
- `method`: String – Phương thức (cash | banking)
- `amount`: Number – Số tiền
- `status`: String – Trạng thái thanh toán
- `createdAt`, `updatedAt`: Date

4.2.8 Nhận xét thiết kế

Hệ thống sử dụng kết hợp giữa:

- **Reference (tham chiếu)**: giữa các collection (User, Product, Order, Table)
- **Embedded document (nhúng)**: OrderItem trong Order

Cách thiết kế này giúp:

- Tăng hiệu năng truy vấn
- Giảm số lần join dữ liệu
- Phù hợp với đặc điểm của MongoDB

4.3 Thiết kế API (API Design)

Hệ thống cung cấp các API theo chuẩn RESTful để phục vụ các chức năng chính. Các API sử dụng giao thức HTTP, dữ liệu trao đổi dưới dạng JSON và được tổ chức theo từng nhóm chức năng rõ ràng.

Tiền tố chung của hệ thống:

Base URL: /api/v1

4.3.1 API xác thực (Authentication API)

- **Đăng ký**
 - **POST** /api/v1/auth/register

- Mô tả: Tạo tài khoản người dùng mới
- **Đăng nhập**
 - **POST** /api/v1/auth/login
 - Mô tả: Xác thực người dùng và trả về JWT token
- **Lấy thông tin cá nhân**
 - **GET** /api/v1/auth/profile
 - Yêu cầu: Token
 - Mô tả: Lấy thông tin người dùng đang đăng nhập

4.3.2 API quản lý người dùng (User API)

- **Lấy thông tin cá nhân**
 - **GET** /api/v1/users/me
- **Cập nhật thông tin cá nhân**
 - **PUT** /api/v1/users/me
- **Lấy danh sách người dùng (Admin)**
 - **GET** /api/v1/users
- **Tạo người dùng**
 - **POST** /api/v1/users
- **Cập nhật người dùng**
 - **PUT** /api/v1/users/:id
- **Xóa người dùng**
 - **DELETE** /api/v1/users/:id

4.3.3 API quản lý sản phẩm (Product API)

- **Lấy danh sách sản phẩm (Public)**
 - **GET** /api/v1/products
- **Lấy chi tiết sản phẩm**
 - **GET** /api/v1/products/:id
- **Thêm sản phẩm (Admin)**

- **POST** /api/v1/products
- **Cập nhật sản phẩm**
 - **PUT** /api/v1/products/:id
- **Xóa sản phẩm**
 - **DELETE** /api/v1/products/:id
- **Bật/Tắt sản phẩm**
 - **PATCH** /api/v1/products/:id/toggle

4.3.4 API quản lý danh mục (Category API)

- **Lấy danh mục (Public)**
 - **GET** /api/v1/categories
- **Lấy tất cả danh mục (Admin)**
 - **GET** /api/v1/categories/admin
- **Thêm danh mục**
 - **POST** /api/v1/categories
- **Cập nhật danh mục**
 - **PUT** /api/v1/categories/:id
- **Xóa danh mục**
 - **DELETE** /api/v1/categories/:id

4.3.5 API quản lý bàn (Table API)

- **Lấy danh sách bàn**
 - **GET** /api/v1/tables
- **Lấy bàn theo mã QR**
 - **GET** /api/v1/tables/code/:code
- **Tạo bàn**
 - **POST** /api/v1/tables
- **Cập nhật bàn**
 - **PUT** /api/v1/tables/:id
- **Cập nhật trạng thái bàn**

- **PATCH** /api/v1/tables/:id/status

4.3.6 API quản lý đơn hàng (Order API)

- **Tạo đơn hàng (QR / Staff / App)**

- **POST** /api/v1/orders

- **Lấy đơn hàng của người dùng**

- **GET** /api/v1/orders/me

- **Lấy danh sách đơn hàng (Admin/Staff)**

- **GET** /api/v1/orders

- **Lấy chi tiết đơn hàng**

- **GET** /api/v1/orders/:id

- **Cập nhật trạng thái đơn**

- **PATCH** /api/v1/orders/:id/status

- **Thanh toán đơn hàng**

- **PATCH** /api/v1/orders/:id/pay

- **Xóa đơn hàng**

- **DELETE** /api/v1/orders/:id

4.3.7 API thanh toán (Payment API)

- **Tạo thanh toán**

- **POST** /api/v1/payments

- **Lấy danh sách thanh toán**

- **GET** /api/v1/payments

- **Lấy chi tiết thanh toán**

- **GET** /api/v1/payments/:id

4.3.8 API thống kê (Analytics API)

- **Doanh thu theo ngày**

- **GET** /api/v1/analytics/revenue/day

- **Doanh thu theo tháng**

- **GET** /api/v1/analytics/revenue/month

- **Sản phẩm bán chạy**

- **GET** /api/v1/analytics/best-seller

- **Tổng quan hệ thống**

- **GET** /api/v1/analytics/summary

4.3.9 Bảo mật API

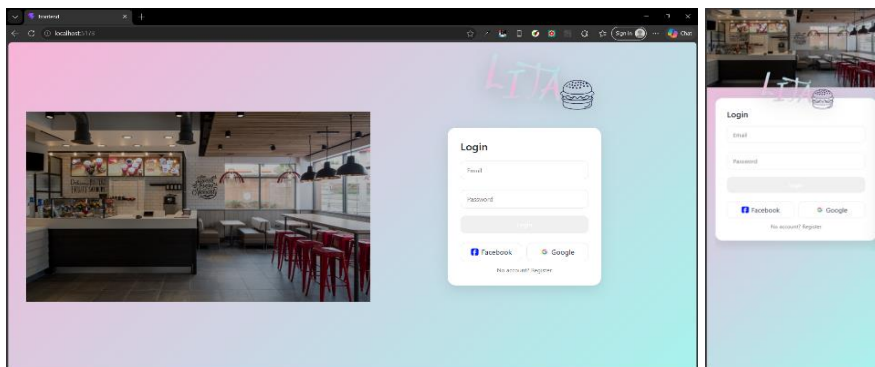
Hệ thống sử dụng JWT (JSON Web Token) để xác thực người dùng.

- Token được gửi qua header:
Authorization: Bearer {token}
- Một số API yêu cầu phân quyền:
 - Admin
 - Staff
 - Customer

4.4 Thiết kế giao diện người dùng (UI Design)

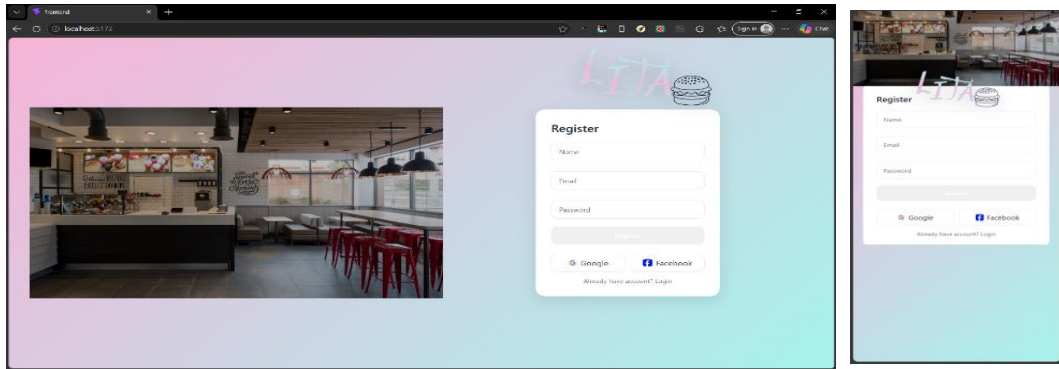
Giao diện người dùng được thiết kế theo tiêu chí đơn giản, trực quan và dễ sử dụng cho tất cả các đối tượng.

4.4.1 Trang Đăng nhập & đăng ký (Auth)



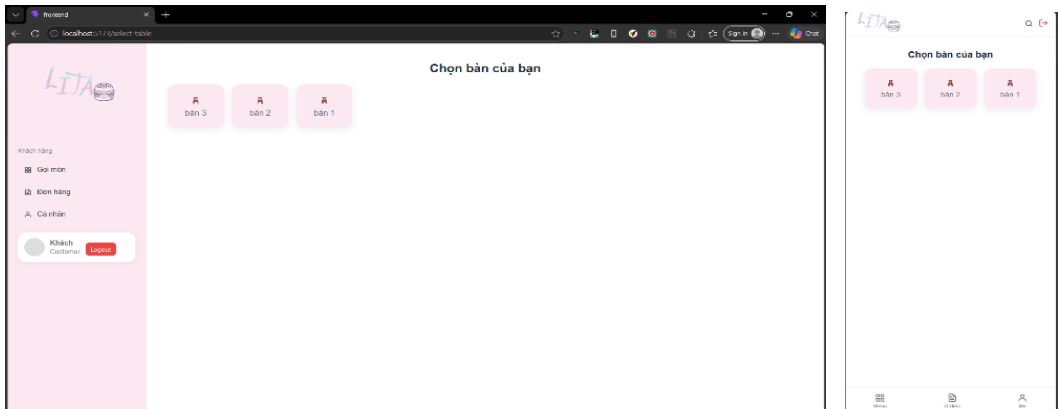
Hình 4.1 Giao diện đăng nhập

Giao diện đăng nhập cho phép người dùng nhập email và mật khẩu để truy cập hệ thống. Sau khi xác thực thành công, hệ thống sẽ điều hướng đến giao diện tương ứng với vai trò Admin hoặc Staff.

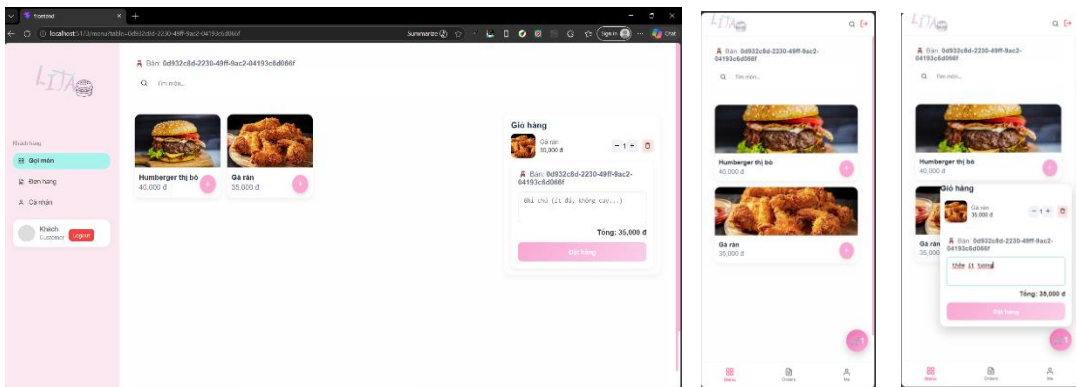


Hình 4.2 Giao diện đăng ký

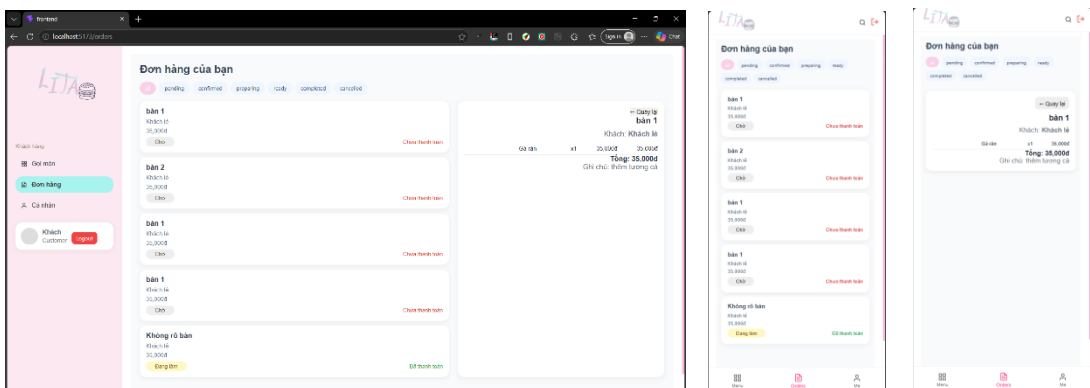
4.4.1 Trang Khách hàng (Customer)



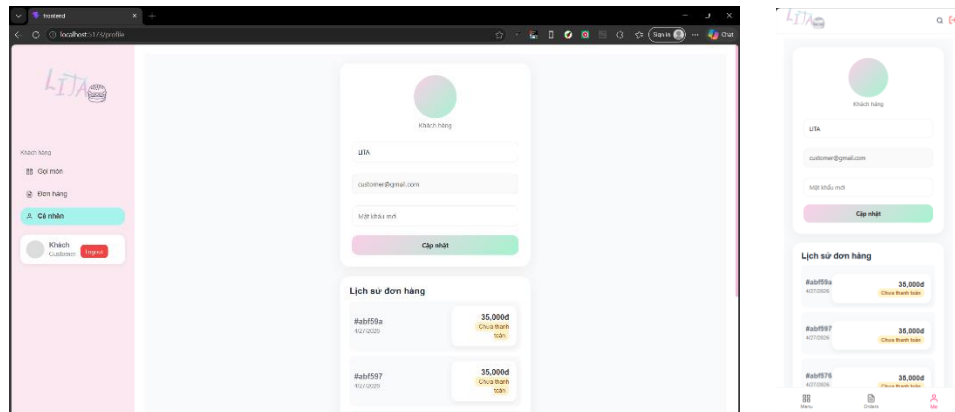
Hình 4.3 Giao diện chọn bàn (theo luồng không QR)



Hình 4.4 giao diện gọi món

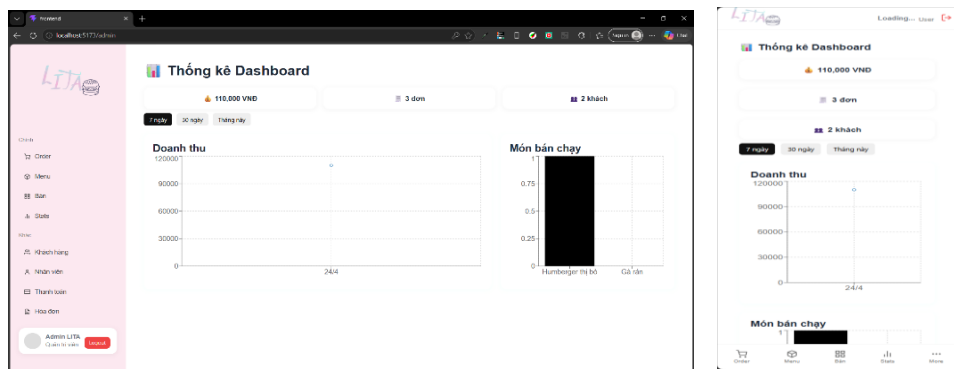


Hình 4.5 Giao diện theo dõi đơn hàng

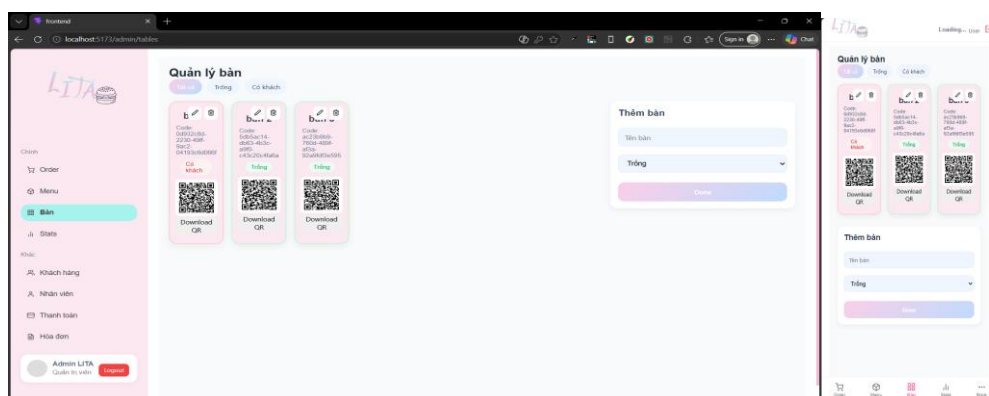


Hình 4.6 Giao diện Profile và lịch sử

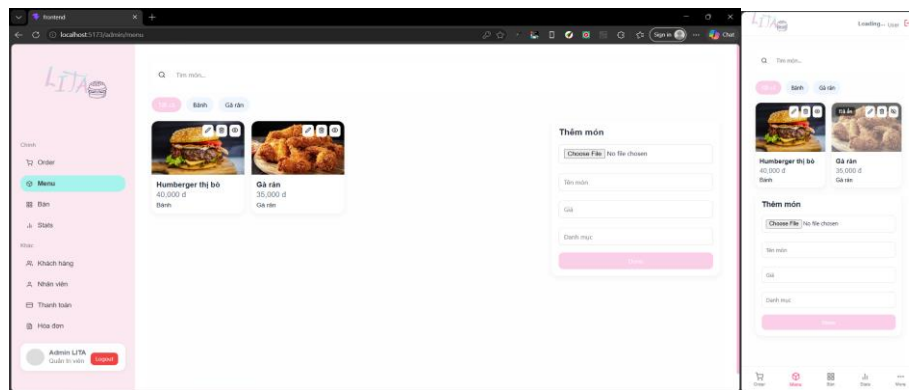
4.4.2 Trang quản trị (Admin)



Hình 4.7 Giao diện DashBoard.

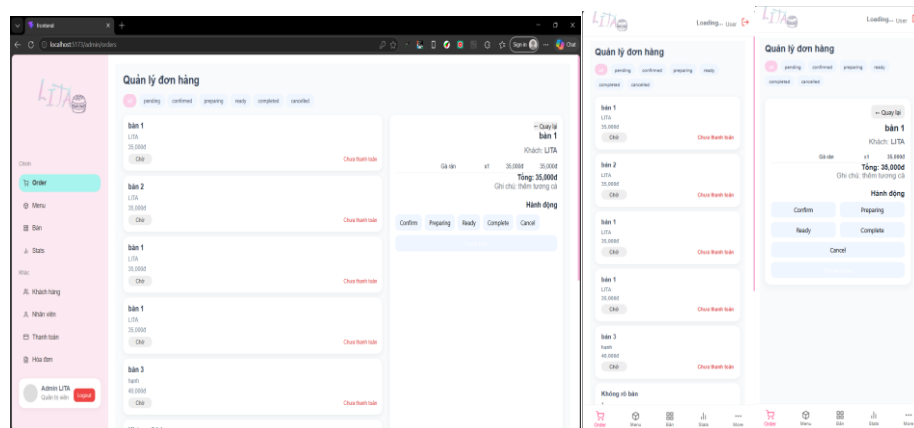


Hình 4.8 Giao diện quản lý bàn.

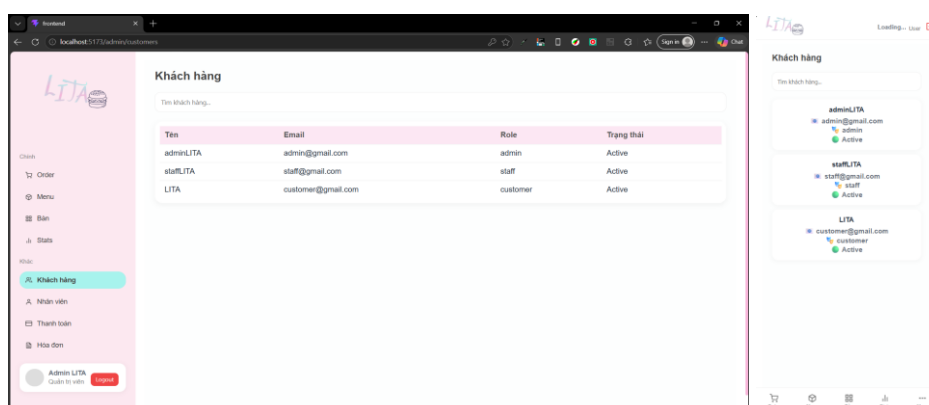


Hình 4.9 Giao diện quản lý Menu

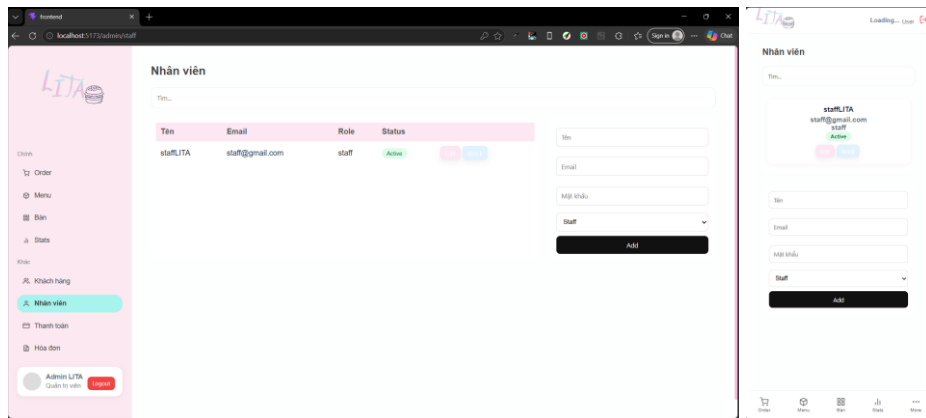
Giao diện quản lý Menu cho phép quản trị viên thêm, sửa, xóa món ăn, cập nhật giá bán và trạng thái món ăn.



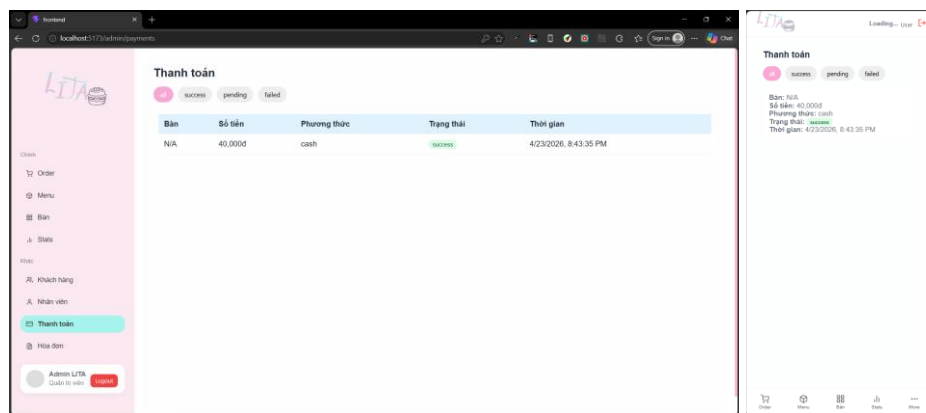
Hình 4.10 Giao diện quản lý đơn hàng



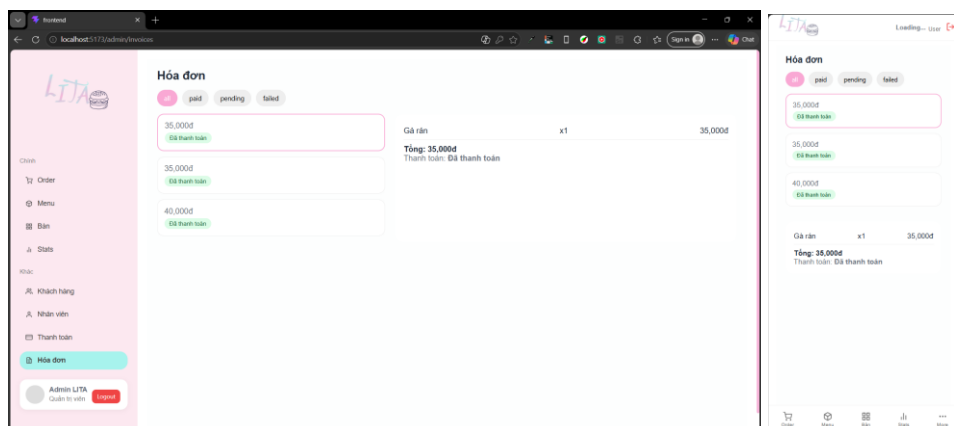
Hình 4.11 Giao diện quản lý khách hàng



Hình 4.12 Giao diện quản lý nhân viên

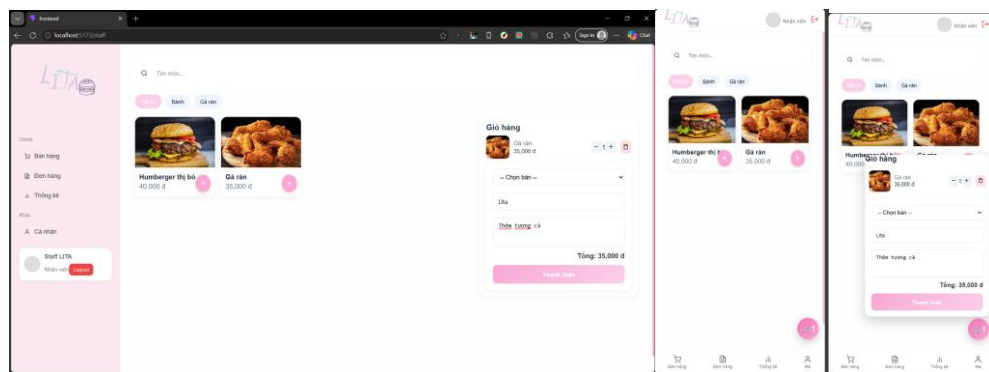


Hình 4.13 Giao diện quản lý thanh toán

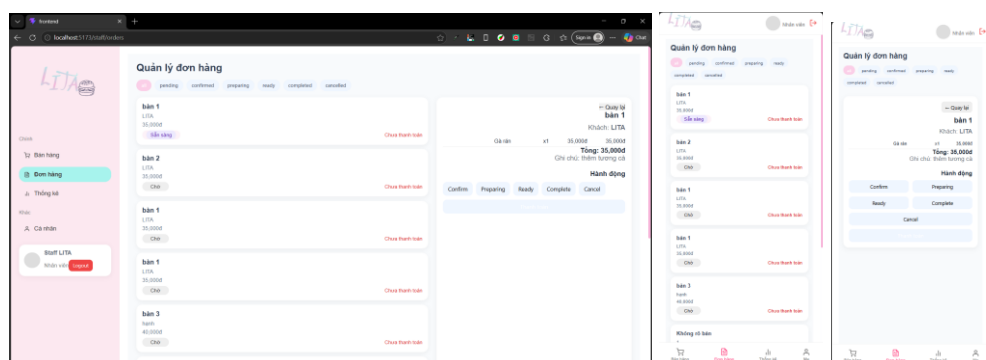


Hình 4.14 Giao diện quản lý hóa đơn

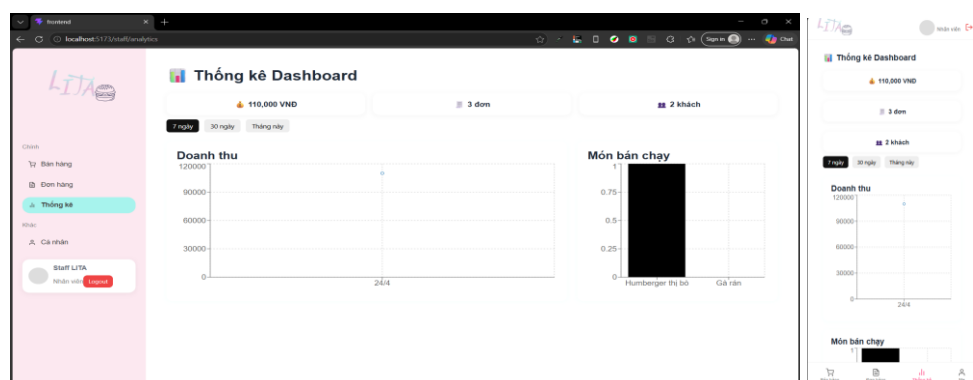
4.4.3 Trang Staff



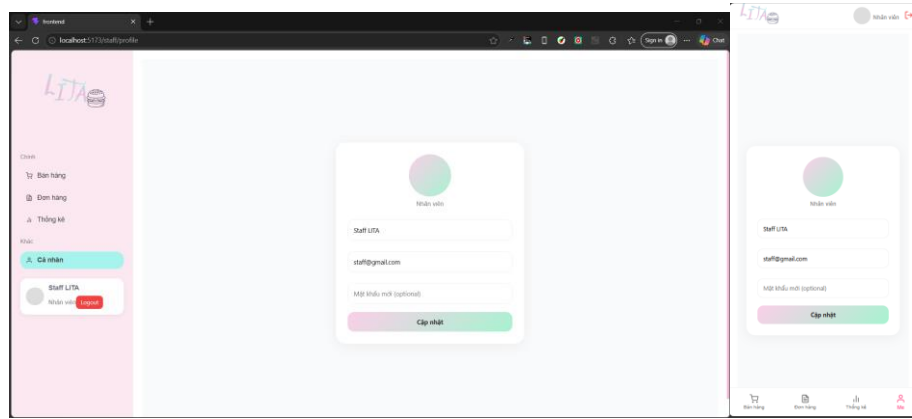
Hình 4.15 Giao diện quản lý POS



Hình 4.16 Giao diện quản lý đơn hàng staff



Hình 4.17 Giao diện thống kê Dashboard Staff



Hình 4.18 Giao diện cá nhân Staff

4.5 Tổng kết chương

Chương này đã trình bày chi tiết thiết kế hệ thống từ kiến trúc tổng thể, cơ sở dữ liệu, API đến giao diện người dùng. Việc tổ chức hệ thống theo mô hình Client – Server kết hợp RESTful API cùng với thiết kế dữ liệu và giao diện hợp lý giúp đảm bảo hệ thống hoạt động hiệu quả, dễ bảo trì và có khả năng mở rộng trong tương lai.

CHƯƠNG 5: CÀI ĐẶT VÀ TRIỂN KHAI

5.1 Môi trường và công nghệ sử dụng

Để hệ thống hoạt động ổn định và đảm bảo khả năng phát triển, đề tài sử dụng các công nghệ và môi trường phát triển phổ biến hiện nay.

5.1.1 Môi trường phát triển

- **Hệ điều hành:** Windows 10 hoặc tương đương
- **Trình soạn thảo mã nguồn:** Visual Studio Code
- **Trình duyệt:** Google Chrome

5.1.2 Công nghệ sử dụng

- **Node.js:** Phiên bản đề xuất từ **v18.x trở lên** để đảm bảo hỗ trợ đầy đủ các tính năng hiện đại và hiệu năng ổn định.
- **MongoDB:** Sử dụng để lưu trữ dữ liệu hệ thống, có thể cài đặt local hoặc sử dụng MongoDB Atlas (cloud).
- **NPM (Node Package Manager):** Quản lý thư viện và dependency cho dự án.

5.2 Cài đặt hệ thống

Quá trình cài đặt hệ thống được thực hiện theo các bước đơn giản, đảm bảo người dùng hoặc người chấm có thể dễ dàng chạy thử dự án.

5.2.1 Chuẩn bị môi trường

- Cài đặt Node.js
- Cài đặt MongoDB (hoặc tạo tài khoản MongoDB Atlas)
- Clone source code từ repository (GitHub)

5.2.2 Cài đặt Backend

Di chuyển đến thư mục backend và thực hiện:

```
npm install
```

Mô tả:

Cài đặt tất cả các thư viện cần thiết như Express, Mongoose, JWT, bcrypt,...

Tiếp theo, chạy server:

```
npm run dev
```

Kết quả:

- Server khởi động thành công tại địa chỉ: `http://localhost:5000`

- Kết nối cơ sở dữ liệu MongoDB

5.2.3 Cài đặt Frontend

Di chuyển đến thư mục frontend và thực hiện:

```
npm install
```

```
npm run dev
```

Kết quả:

- Giao diện web chạy tại: `http://localhost:5173`
- Có thể truy cập từ trình duyệt

5.3 Cấu hình hệ thống

Hệ thống sử dụng file `.env` để cấu hình các biến môi trường quan trọng:

```
PORT=5000
```

```
MONGO_URI=mongodb+srv://LITA
```

```
JWT_SECRET=your_secret_key
```

Giải thích:

- `PORT`: Cổng chạy server
- `MONGO_URI`: Đường dẫn kết nối database
- `JWT_SECRET`: Khóa bảo mật dùng để mã hóa token

5.4 Dữ liệu mẫu (Seed Data)

Để thuận tiện cho việc kiểm thử và demo hệ thống, đề tài sử dụng dữ liệu mẫu (seed data).

5.4.1 Nội dung dữ liệu mẫu

- Danh sách món ăn (Product)
- Danh sách bàn (Table)
- Tài khoản người dùng (Admin, Staff, Customer)

5.4.2 Cách thực hiện

Có thể sử dụng script seed hoặc thêm thủ công thông qua API:

```
node seed.js
```

Kết quả:

- Database được khởi tạo với dữ liệu ban đầu

- Có thể đăng nhập và sử dụng hệ thống ngay

5.5 Kiểm thử sau cài đặt

Sau khi cài đặt thành công, cần kiểm tra các chức năng chính:

- Truy cập trang menu bằng QR
- Tạo đơn hàng
- Nhân viên xử lý đơn
- Thực hiện thanh toán
- Admin quản lý menu và bàn

Kết quả mong đợi:

- Hệ thống hoạt động ổn định
- Không phát sinh lỗi nghiêm trọng

5.6 Các đoạn mã nguồn tiêu biểu

5.6.1 Xác thực và phân quyền người dùng

Đăng ký tài khoản và tạo JWT

Mô tả: Hệ thống kiểm tra email đã tồn tại hay chưa trước khi tạo tài khoản mới. Sau khi đăng ký thành công, hệ thống sinh JWT Token chứa thông tin định danh và quyền của người dùng.

```
10     const existingUser = await User.findOne({ email });
11     if (existingUser) {
12         throw new Error("Email already exists");
13     }
14
15     const user = await User.create({
16         name,
17         email,
18         password, // 🔥 FIX
19         role: "customer",
20     });
21
22     const token = generateToken({
23         id: user._id,
24         role: user.role,
25     });
```

Hình 5.1 Đoạn code đăng ký tài khoản tạo JWT

Xác thực đăng nhập

Mô tả: Khi đăng nhập, hệ thống kiểm tra sự tồn tại của tài khoản, so sánh mật khẩu bằng bcrypt và xác minh trạng thái hoạt động của tài khoản trước khi cho phép truy cập.

```
42     const user = await User.findOne({ email });
43     console.log("USER:", user);
44
45     if (!user) throw new Error("User not found");
46
47     const isMatch = await bcrypt.compare(password, user.password);
48     console.log("MATCH:", isMatch);
49
50     if (!isMatch) throw new Error("Invalid password");
51
52     if (!user.isActive) throw new Error("Account is disabled");
53
```

Hình 5.2 Đoạn code xác thực đăng nhập

5.6.2 Xử lý tạo đơn hàng

Kiểm tra dữ liệu đơn hàng

Mô tả: Hệ thống thực hiện kiểm tra tính hợp lệ của đơn hàng trước khi xử lý, bao gồm nguồn tạo đơn, danh sách món ăn, thông tin khách hàng và điều kiện xác thực đối với người dùng đặt món trực tuyến.

```
16   if (!["staff", "qr", "app"].includes(source)) {
17     throw new Error("Invalid order source");
18   }
19
20   if (!items?.length) {
21     throw new Error("Order items are required");
22   }
23
24   let tableDoc;
25
26   // ===== STAFF =====
27   if (source === "staff") {
28     tableDoc = await Table.findById(table);
29   }
30
31   // ===== QR / APP =====
32   if (source === "qr" || source === "app") {
33     tableDoc = await Table.findOne({ code: table });
34   }
35
36   // ===== VALIDATE =====
37   if (!tableDoc) throw new Error("Table not found");
38
39   // ===== STAFF RULE =====
40   if (source === "staff" && !guestName) {
41     throw new Error("Guest name is required for staff order");
42   }
```

Hình 5.3 Đoạn code kiểm tra dữ liệu đơn hàng

Kiểm tra và cập nhật tồn kho

Mô tả: Trước khi tạo đơn hàng, hệ thống kiểm tra số lượng tồn kho của từng món ăn. Sau khi đơn hàng hợp lệ, số lượng tồn được cập nhật tự động và món ăn sẽ được chuyển sang trạng thái hết hàng khi tồn kho bằng 0.

```

70  // 🔥 CHECK STOCK
71  if (product.stockQuantity <= 0) {
72    throw new Error(`${product.name} is out of stock`);
73  }
74
75  if (quantity > product.stockQuantity) {
76    throw new Error(`${product.name} only has ${product.stockQuantity} left`);
77  }
78
79  // 🔥 TRỪ KHO
80  product.stockQuantity -= quantity;
81
82  // 🔥 AUTO HẾT HÀNG    "HÀNG": Unknown word.
83  if (product.stockQuantity === 0) {
84    product.isAvailable = false;
85  }
86
87
88
89  await product.save();

```

Hình 5.4 Đoạn code kiểm tra và cập nhật tồn kho

5.6.3 Quản lý bàn và mã QR

Sinh mã QR cho bàn ăn

Mô tả: Hệ thống tự động tạo mã định danh duy nhất (UUID) cho mỗi bàn ăn. Sau đó hệ thống sinh mã QR tương ứng chứa đường dẫn truy cập đến giao diện gọi món của bàn. Mã QR được lưu vào cơ sở dữ liệu và sử dụng trong quá trình khách hàng quét mã để đặt món trực tiếp.

```

26  // CREATE
27  export const createTableService = async (data) => {
28    const code = uuidv4();    // "uuidv": Unknown word.
29
30    const url = `${ENV.CLIENT_URL}/menu?table=${code}`;
31    const qrCode = await QRCode.toDataURL(url);
32
33    const table = await Table.create({
34      name: data.name,        // chỉ nhận name        "nhận": Unknown word.
35      status: "available",    // backend quyết định    "quyết": Unknown word.
36      code,
37      qrCode,
38    });
39    return table;
40  };

```

Hình 5.5 Đoạn code sinh mã qr bàn ăn

Tra cứu bàn từ mã QR

Mô tả: Khi khách hàng quét mã QR, hệ thống sử dụng mã định danh của bàn để truy xuất thông tin bàn tương ứng, từ đó hiển thị thực đơn và thực hiện quá trình gọi món.

```
6   export const getTableByCodeService = async (code) => {  
7     const table = await Table.findOne({ code });  
8     if (!table) throw new Error("Table not found");  
9     return table;  
10  };
```

Hình 5.6 Đoạn code tra cứu mã QR

5.6.4 Quản lý thực đơn

Kiểm tra trạng thái món ăn trước khi gọi món

Mô tả: Trước khi thêm món ăn vào đơn hàng, hệ thống kiểm tra trạng thái khả dụng của sản phẩm nhằm đảm bảo khách hàng chỉ có thể đặt những món đang được phục vụ.

```
if (!product.isAvailable) {  
    throw new Error(`${product.name} is not available`);  
}
```

Hình 5.7 Đoạn code kiểm tra trạng thái món ăn

Kiểm tra số lượng tồn kho

Mô tả: Sau khi đơn hàng được tạo thành công, hệ thống tự động cập nhật số lượng tồn kho của món ăn. Khi số lượng tồn bằng 0, món ăn sẽ được chuyển sang trạng thái hết hàng và không còn hiển thị cho khách hàng đặt món.

```
product.stockQuantity -= quantity;  
  
if (product.stockQuantity === 0) {  
    product.isAvailable = false;  
}  
  
await product.save();
```

Hình 5.8 Đoạn code kiểm tra số lượng tồn kho

5.6.5 Thanh toán đơn hàng

Kiểm tra trạng thái thanh toán đơn hàng

Mô tả: Trước khi thực hiện thanh toán, hệ thống kiểm tra sự tồn tại của đơn hàng và trạng thái thanh toán nhằm tránh việc thanh toán trùng lặp hoặc xử lý dữ liệu không hợp lệ.

```
const order = await Order.findById(orderId);

if (!order) {
    throw new Error("Order not found");
}

if (order.paymentStatus === "paid") {
    throw new Error("Order already paid");
}
```

Hình 5.9 Đoạn code trạng thái thanh toán đơn hàng

Tạo giao dịch thanh toán

Mô tả: Khi khách hàng hoàn tất thanh toán, hệ thống tạo bản ghi giao dịch bao gồm phương thức thanh toán, nhà cung cấp dịch vụ thanh toán và giá trị đơn hàng để phục vụ việc quản lý và đối soát sau này.

```
19  const payment = await Payment.create({
20      order: order._id,
21      method,
22      provider,
23      amount: order.totalAmount,
24      status: "success",
25  });
```

Hình 5.10 Đoạn code tạo giao dịch thanh toán

Cập nhật trạng thái đơn hàng sau thanh toán

Mô tả: Sau khi thanh toán thành công, hệ thống tự động cập nhật trạng thái đơn hàng sang đã thanh toán và hoàn thành. Đồng thời số lượng bán của từng món ăn được ghi nhận để phục vụ chức năng thống kê và xác định các sản phẩm bán chạy.

```
// 📌 update order

order.paymentStatus = "paid";
order.status = "completed";

await order.save();

// sold count

for (const item of order.items) {
  await Product.findByIdAndUpdate(
    item.product,
    {
      $inc: {
        soldCount: item.quantity,
      },
    },
  );
}
```

Hình 5.11 Đoạn code cập nhật trạng thái đơn hàng sau thanh toán

5.6.6 Thống kê doanh thu

Thống kê doanh thu theo ngày

Mô tả: Hệ thống sử dụng Aggregation Pipeline của MongoDB để tổng hợp doanh thu theo từng ngày dựa trên các đơn hàng đã thanh toán thành công. Kết quả được sắp xếp theo trình tự thời gian để phục vụ hiển thị biểu đồ doanh thu.

```
return await Order.aggregate([
  {
    $match: matchStage,
  },
  {
    $group: {
      _id: {
        day: { $dayOfMonth: "$createdAt" },
        month: { $month: "$createdAt" },
        year: { $year: "$createdAt" },
      },
      totalRevenue: {
        $sum: "$totalAmount",
      },
    },
  },
  {
    $sort: {
      "_id.year": 1,
      "_id.month": 1,
      "_id.day": 1,
    },
  },
]);
```

Hình 5.12 Đoạn code thống kê doanh thu theo ngày

Thống kê doanh thu theo tháng

Mô tả: Hệ thống thực hiện tổng hợp dữ liệu doanh thu và số lượng đơn hàng theo từng tháng, giúp nhà quản lý đánh giá hiệu quả kinh doanh trong các giai đoạn khác nhau.

```
54 // ===== REVENUE BY MONTH =====
55 export const getRevenueByMonthService = async () => {
56   return await Order.aggregate([
57     {
58       $match: {
59         paymentStatus: "paid",
60       },
61     },
62     {
63       $group: {
64         _id: {
65           month: { $month: "$createdAt" },
66           year: { $year: "$createdAt" },
67         },
68         totalRevenue: { $sum: "$totalAmount" },
69         totalOrders: { $sum: 1 },
70       },
71     },
72     {
73       $sort: {
74         "_id.year": 1,
75         "_id.month": 1,
76       },
77     },
78   ]);
79 };
```

Hình 5.13 Đoạn code thống kê doanh thu theo tháng

Tổng hợp dữ liệu Dashboard

Mô tả: Hệ thống tổng hợp các chỉ số quan trọng phục vụ Dashboard quản trị bao gồm tổng doanh thu, tổng số đơn hàng và số lượng khách hàng đã phát sinh giao dịch. Các dữ liệu này được sử dụng để hiển thị báo cáo tổng quan theo thời gian thực.

```
export const getBestSellerService = async () => {
  return await Product.find()
    .sort({ soldCount: -1 })
    .limit(10)
    .select("name soldCount price image");
};
```

Hình 5.14 Đoạn code tổng hợp dữ liệu

5.7 Tổng kết chương

Chương này đã trình bày chi tiết quá trình cài đặt và triển khai hệ thống từ môi trường phát triển, cấu hình, cài đặt các thành phần cho đến việc tạo dữ liệu mẫu và kiểm thử. Với quy trình đơn giản và rõ ràng, hệ thống có thể dễ dàng được triển khai và sử dụng trong thực tế hoặc phục vụ cho mục đích đánh giá đồ án.

CHƯƠNG 6: KIỂM THỬ HỆ THỐNG

Kiểm thử hệ thống là một bước quan trọng nhằm đảm bảo ứng dụng hoạt động đúng theo yêu cầu, ổn định và đáp ứng tốt trong môi trường thực tế. Trong đề tài này, quá trình kiểm thử được thực hiện trên các chức năng chính của hệ thống web quản lý cửa hàng thức ăn nhanh, bao gồm kiểm thử chức năng, kiểm thử khả dụng, theo dõi lỗi và kiểm thử hiệu năng.

6.1 Kiểm thử chức năng (Functional Testing)

Kiểm thử chức năng được thực hiện nhằm xác minh rằng toàn bộ các chức năng chính của hệ thống hoạt động đúng theo yêu cầu đã thiết kế. Việc kiểm thử dựa trên các luồng nghiệp vụ thực tế, kết hợp với việc kiểm tra trực tiếp thông qua API và giao diện người dùng.

Hệ thống được kiểm thử dựa trên các module chính sau:

- Quản lý người dùng (đăng ký, đăng nhập, phân quyền)
- Quản lý thực đơn (category, product)
- Quản lý bàn (table + QR)
- Quản lý đơn hàng (order)
- Thanh toán (payment)
- Thống kê (analytics)

Các test case được xây dựng dựa trên logic trong service layer – nơi xử lý nghiệp vụ chính của hệ thống.

6.1.1 Bảng test case chi tiết

Bảng 6.1 Bảng Test case tiêu biểu

STT	Chức năng	Input	Expected Output
1	Tạo đơn hàng (QR)	Table code hợp lệ + danh sách món	Tạo order thành công, status = pending
2	Tạo đơn hàng (staff)	Table ID + guestName + items	Order được tạo, lưu guestName
3	Tạo đơn thiếu items	items = []	Trả lỗi "Order items are required"

STT	Chức năng	Input	Expected Output
4	Tạo đơn sai source	source không thuộc ["staff", "qr", "app"]	Trả lỗi "Invalid order source"
5	Xem menu	GET /products	Trả danh sách sản phẩm isAvailable = true
6	Tìm kiếm món	keyword = "trà sữa"	Trả sản phẩm phù hợp
7	Lọc theo category	category ID	Trả đúng nhóm món
8	Thêm món (admin)	name, price, category	Tạo product thành công
9	Ẩn/hiện món	toggleProduct	isAvailable đảo trạng thái
10	Tạo bàn	name = "Bàn 1"	Sinh QR + code + status available
11	Lấy bàn theo QR	code	Trả đúng thông tin bàn
12	Thanh toán	orderId + method	payment tạo thành công
13	Thanh toán lại	order đã paid	Trả lỗi "Order already paid"
14	Đăng ký	email chưa tồn tại	Tạo user + token
15	Đăng ký trùng email	email đã tồn tại	Lỗi "Email already exists"
16	Đăng nhập	email + password đúng	Trả token
17	Đăng nhập sai password	password sai	Lỗi "Invalid password"
18	Phân quyền	staff gọi API admin	Bị từ chối
19	Xem đơn của user	GET /my-orders	Trả đúng order theo customer
20	Xóa order	orderId hợp lệ	Order bị xóa + bàn available

6.1.2 Phân tích kiểm thử theo từng module

1. Kiểm thử chức năng tạo đơn hàng

Chức năng tạo đơn hàng được kiểm thử dựa trên service:

- createOrderService

Các trường hợp kiểm thử chính:

- Kiểm tra nguồn đơn (source)
- Kiểm tra tồn tại bàn (Table)
- Kiểm tra đăng nhập (QR/app)
- Kiểm tra danh sách món (items)
- Kiểm tra trạng thái món (isAvailable)
- Tính toán totalAmount

Kết quả:

- Đơn hàng được tạo chính xác
- Tổng tiền được tính đúng
- Dữ liệu được populate đầy đủ (table, product, user)
- Bàn chuyển sang trạng thái occupied khi đặt bằng QR

2. Kiểm thử chức năng thanh toán

Thực hiện qua:

- createPaymentService

Kiểm thử:

- Thanh toán đơn chưa thanh toán
- Thanh toán lại đơn đã paid
- Kiểm tra cập nhật:
 - order.paymentStatus = paid
 - order.status = completed
 - product.soldCount tăng

Kết quả:

- Thanh toán thành công
- Không cho thanh toán trùng
- Dữ liệu thống kê chính xác

3. Kiểm thử quản lý thực đơn (Product + Category)

Các chức năng:

- Thêm / sửa / xóa món
- Tìm kiếm + lọc
- Toggle trạng thái món

Kiểm thử logic:

- Nếu isAvailable = false → không hiển thị cho khách
- Tự động tạo category nếu chưa tồn tại

Kết quả:

- Menu hiển thị đúng cho user
- Admin quản lý linh hoạt

4. Kiểm thử quản lý bàn (Table + QR)

Thông qua:

- createTableService
- getTableByCodeService

Kiểm thử:

- Tạo bàn → sinh QR code
- Truy cập QR → lấy đúng bàn
- Khi có order → chuyển sang occupied

Kết quả:

- QR hoạt động chính xác
- Mapping bàn ổn định

5. Kiểm thử xác thực & phân quyền

Thông qua:

- registerService
- loginService

Kiểm thử:

- Hash password bằng bcrypt

- So sánh password khi login
- Token JWT

Phân quyền:

- Admin / Staff / Customer

Kết quả:

- Đăng nhập hoạt động đúng
- Không thể truy cập trái phép
- Bảo mật đảm bảo

6. Kiểm thử API đơn hàng

Các API:

- GET /orders
- GET /orders/:id
- PUT /orders/:id
- DELETE /orders/:id

Kiểm thử:

- Filter theo status
- Populate dữ liệu liên quan
- Xóa order → reset bàn

Kết quả:

- API trả đúng dữ liệu
- Không lỗi logic

6.1.3 Kết quả kiểm thử

Sau quá trình kiểm thử chức năng toàn diện, hệ thống đạt được các kết quả sau:

- Tất cả các chức năng chính hoạt động đúng theo yêu cầu
- Các rule nghiệp vụ (validation) hoạt động chính xác
- Không phát sinh lỗi nghiêm trọng (critical bug)
- Các lỗi nhỏ đã được phát hiện và khắc phục kịp thời
- Hệ thống đảm bảo tính nhất quán dữ liệu (order – payment – product – table)

6.1.4 Nhận xét

Việc thiết kế theo mô hình:

- Controller → Service → Model

giúp:

- Dễ kiểm thử từng lớp
- Tách biệt rõ logic nghiệp vụ
- Dễ mở rộng và bảo trì

6.2 Kiểm thử khả dụng (Usability Testing)

Kiểm thử khả dụng được nhằm đánh giá mức độ thân thiện, dễ sử dụng và hiệu quả của hệ thống đối với người dùng cuối. Trong đề tài này, hệ thống được thiết kế hướng đến hai nhóm người dùng chính:

- Khách hàng (customer): sử dụng QR để đặt món
- Nhân viên / quản trị (staff / admin): sử dụng hệ thống quản lý

Việc kiểm thử khả dụng tập trung vào các luồng nghiệp vụ thực tế đã được triển khai trong hệ thống, bao gồm:

- Truy cập menu qua QR
- Tạo đơn hàng
- Thanh toán
- Quản lý menu, bàn, đơn hàng (admin)

6.2.1 Phương pháp kiểm thử

Kiểm thử được bằng phương pháp User Testing (kiểm thử với người dùng thực) kết hợp quan sát hành vi.

Cách thực hiện:

- Mời người dùng thử nghiệm (sinh viên, bạn bè)
- Không hướng dẫn chi tiết cách sử dụng
- Yêu cầu thực hiện các tác vụ:
 - Quét QR và đặt món
 - Xem giỏ hàng và chỉnh sửa
 - Thực hiện thanh toán

- Ghi nhận:
 - Thời gian thao tác
 - Số lần thao tác sai
 - Các điểm gây khó hiểu

6.2.2 Kịch bản kiểm thử thực tế

Kịch bản 1: Đặt món qua QR (Customer Flow)

Luồng thực tế:

1. Quét QR bàn → truy cập /menu?table={code}
2. Hệ thống gọi API:
 - getTableByCodeService
 - getProductsService
3. Người dùng:
 - Chọn món
 - Thêm vào giỏ hàng
4. Nhấn “Đặt món”
5. Gửi request:
 - createOrderService (source = "qr")

Các yếu tố được đánh giá:

- Có cần đăng nhập không
- Có hiểu được menu không
- Có dễ thêm món không
- Có nhận biết được trạng thái đơn không

Kết quả:

- Người dùng không cần hướng dẫn vẫn hoàn thành được luồng
- Việc sử dụng QR giúp giảm bước nhập liệu (không cần chọn bàn)
- Giao diện menu rõ ràng do:
 - Có phân loại category
 - Có trạng thái isAvailable

Kịch bản 2: Nhân viên tạo đơn (Staff Flow)

Luồng thực tế:

1. Nhân viên chọn bàn
2. Nhập guestName
3. Chọn món
4. Gửi request:
 - createOrderService (source = "staff")

Kiểm thử các điểm:

- Có dễ nhập thông tin khách không
- Có nhầm lẫn giữa bàn và khách không
- Có phát hiện lỗi thiếu guestName không

Kết quả:

- Hệ thống bắt buộc guestName giúp tránh nhầm đơn
- Thông báo lỗi rõ ràng:
 - "Guest name is required for staff order"
- Luồng thao tác đơn giản, ít bước

Kịch bản 3: Thanh toán

Luồng thực tế:

1. Chọn đơn hàng
2. Nhấn thanh toán
3. Gọi:
 - createPaymentService

Hệ thống xử lý:

- Kiểm tra:
 - Order tồn tại
 - Order chưa thanh toán
- Cập nhật:
 - paymentStatus = paid

- status = completed
- soldCount của product

Đánh giá usability:

- Người dùng dễ hiểu trạng thái đơn:
 - unpaid → paid
- Không xảy ra nhầm lẫn thanh toán lặp

Kết quả:

- Thao tác nhanh (1 bước)
- Không gây lỗi trùng thanh toán

Kịch bản 4: Quản lý admin

Chức năng kiểm thử:

- Thêm sản phẩm
- Tạo bàn
- Xem đơn hàng
- Thống kê

Các API liên quan:

- createProductService
- createTableService
- getOrdersService
- getDashboardSummaryService

Đánh giá:

- Dữ liệu hiển thị rõ ràng (populate đầy đủ)
- Thao tác CRUD đơn giản
- Không cần thao tác phức tạp

6.2.3 Các tiêu chí đánh giá

Bảng 6.2 Bảng đánh giá

Tiêu chí	Mô tả	Kết quả
Dễ học (Learnability)	Người dùng mới có thể sử dụng ngay	Đạt
Hiệu quả (Efficiency)	Thực hiện nhanh, ít bước	Đạt
Tỷ lệ lỗi (Error Rate)	Ít thao tác sai	Thấp
Ghi nhớ (Memorability)	Dễ dùng lại sau thời gian	Đạt
Hài lòng (Satisfaction)	Trải nghiệm tốt	Tốt

6.2.4 Các vấn đề phát hiện và cải tiến

Dựa trên quá trình test thực tế, một số điểm đã được cải tiến:

1. Giỏ hàng chưa rõ ràng

- Vấn đề: Người dùng khó thấy tổng tiền
- Giải pháp:
 - Hiện thị totalAmount rõ ràng (frontend + backend đã có field)

2. Thiếu feedback sau khi đặt món

- Vấn đề: Người dùng không chắc đã đặt thành công
- Giải pháp:
 - Hiện thị thông báo success
 - Load lại danh sách đơn

3. Trạng thái bàn chưa trực quan

- Vấn đề: Không biết bàn đang có khách
- Giải pháp:
 - Sử dụng table.status = occupied
 - Hiện thị màu trạng thái

4. Kiểm tra lỗi chưa thân thiện

- Ví dụ:

- "Product not available"
- Giải pháp:
 - Mapping sang thông báo dễ hiểu hơn cho UI

6.2.5 Kết quả kiểm thử khả dụng

Sau quá trình kiểm thử, hệ thống đạt được:

- Người dùng có thể sử dụng không cần hướng dẫn
- Luồng QR → Order → Payment hoạt động mượt
- Thời gian thao tác nhanh do:
 - Không cần nhập nhiều dữ liệu
 - Tự động xử lý logic backend
- Hệ thống hạn chế lỗi người dùng nhờ:
 - Validation chặt chẽ trong service

6.2.6 Nhận xét

Hệ thống đạt mức khả dụng cao, đặc biệt ở:

- Trải nghiệm đặt món bằng QR
- Tối giản thao tác người dùng
- Đồng bộ logic giữa frontend và backend

Việc thiết kế các rule trong service như:

- Bắt buộc guestName (staff)
- Bắt buộc login (QR/app)
- Kiểm tra isAvailable
- Không cho thanh toán lại

6.3 Theo dõi lỗi và khắc phục (Bug Tracking)

Trong quá trình phát triển hệ thống, việc phát hiện, theo dõi và xử lý lỗi một cách có hệ thống nhằm đảm bảo chất lượng phần mềm. Hệ thống sử dụng nền tảng GitHub Issues để quản lý toàn bộ vòng đời của lỗi.

6.3.1 Phương pháp quản lý lỗi

Các lỗi được ghi nhận dựa trên:

- Kiểm thử chức năng (Functional Testing)

- Kiểm thử khả dụng (Usability Testing)
- Log hệ thống (console.log trong service/controller)

Mỗi lỗi được lưu với các thông tin:

- Mô tả lỗi
- Bước tái hiện (reproduce)
- Mức độ nghiêm trọng (severity)
- Trạng thái xử lý (open → in progress → closed)

6.3.2 Phân loại mức độ lỗi

Dựa trên hệ thống hiện tại, lỗi được phân loại như sau:

Bảng 6.3 Bảng phân loại lỗi

Mức độ	Mô tả	Ví dụ trong hệ thống
Critical	Lỗi làm hệ thống không hoạt động	Không tạo được order
Major	Lỗi chức năng chính	Thanh toán sai trạng thái
Minor	Lỗi nhỏ, không ảnh hưởng lớn	Hiển thị sai text
Enhancement	Cải tiến UX	Thêm thông báo

6.3.3 Quy trình xử lý lỗi

Quy trình xử lý lỗi trong hệ thống được theo 5 bước:

Bước 1: Phát hiện lỗi

Lỗi được phát hiện thông qua:

- Test API (Postman / frontend)
- Log backend (ví dụ: console.log("PAY ERROR:", err))

Bước 2: Tạo issue

Một issue được tạo trên GitHub với nội dung:

- API bị lỗi (ví dụ: /api/orders)

- Input gây lỗi
- Thông báo lỗi trả về

Bước 3: Xác định nguyên nhân

Dựa trên code service, xác định nguyên nhân gốc.

Ví dụ thực tế từ hệ thống:

Case 1: Tạo order lỗi table

Trong createOrderService:

```
if (source === "qr" || source === "app") {  
  tableDoc = await Table.findOne({ code: table });  
}
```

Nguyên nhân lỗi:

- Frontend gửi sai table (không phải code)
- Backend không tìm thấy table

Case 2: Thanh toán trùng

Trong createPaymentService:

```
if (order.paymentStatus === "paid") {  
  throw new Error("Order already paid");  
}
```

Đây là bug được prevent từ đầu (defensive coding)

Case 3: Lỗi quantity

```
if (quantity <= 0) throw new Error("Invalid quantity");
```

Tránh lỗi logic sai dữ liệu

Bước 4: Sửa lỗi

Các lỗi được xử lý trực tiếp trong service:

- Thêm validation
- Fix logic
- Bổ sung điều kiện kiểm tra

Ví dụ:

- Fix mapping table $\rightarrow$ _id
- Fix validate user khi order QR:

```
if ((source === "qr" || source === "app") && (!user || !user._id)) {  
  throw new Error("Login required for customer order");  
}
```

Bước 5: Kiểm tra lại (Retest)

Sau khi fix:

- Test lại API
- Test lại UI
- Kiểm tra các trường hợp edge-case

6.3.4 Các lỗi tiêu biểu đã xử lý

1. Lỗi không lấy được đơn theo user

- API: /orders/customer
- Nguyên nhân:
 - table=null từ frontend
- Fix:
 - Kiểm tra customer: req.user._id

2. Lỗi table không đúng format

- Nguyên nhân:
 - QR gửi code nhưng backend dùng _id
- Fix:
 - Phân biệt rõ:

staff $\rightarrow$ findById

qr/app $\rightarrow$ findOne({ code })

3. Lỗi thanh toán không đồng bộ

- Trước:
 - Chỉ update paymentStatus
- Sau:

- Update thêm:
 - status = completed
 - soldCount

4. Lỗi duplicate email

- Fix trong registerService:

```
const existingUser = await User.findOne({ email });
if (existingUser) {
  throw new Error("Email already exists");
}
```

6.3.5 Kết quả đạt được

Sau quá trình theo dõi và xử lý lỗi:

- Các lỗi được quản lý tập trung qua GitHub Issues
- Hệ thống có validation đầy đủ ở service layer
- Giảm thiểu lỗi runtime (crash)
- Tăng độ ổn định của hệ thống
- Code trở nên “defensive” hơn (chống lỗi từ đầu vào)

6.3.6 Nhận xét

Việc áp dụng:

- Service layer validation
- Error handling rõ ràng
- Logging trong backend

Giúp hệ thống:

- Dễ debug
- Dễ mở rộng
- Giảm lỗi phát sinh về sau

6.4 Kiểm thử hiệu năng (Performance Testing)

Kiểm thử hiệu năng được nhằm đánh giá khả năng xử lý của hệ thống trong các điều kiện tải khác nhau, đảm bảo hệ thống có thể phục vụ nhiều người dùng đồng thời mà vẫn duy trì hiệu suất ổn định.

Các kịch bản kiểm thử được xây dựng dựa trên các API trọng yếu:

- createOrderService
- createPaymentService
- getProductsService
- getOrdersService

6.4.1 Load Testing

Mô tả

Kiểm tra hệ thống dưới tải bình thường, tương ứng với nhiều người dùng cùng lúc truy cập hệ thống (đặt món, xem menu).

Thực hiện

- Gửi đồng thời nhiều request:
 - POST /orders
 - GET /products
- Mỗi request:
 - Có từ 1–3 items
 - Có kiểm tra product (findById)
 - Tính toán totalAmount

Đây là thao tác nặng DB vì:

- Query nhiều lần (Product.findById)
- Populate sau khi tạo order

Kết quả

- Hệ thống phản hồi ổn định
- Không xảy ra lỗi crash
- Thời gian phản hồi trung bình: ~2–3 giây/request

Phù hợp với:

- Mô hình quán ăn nhỏ / vừa
- Số lượng user đồng thời trung bình

6.4.2 Stress Testing

Mô tả

Kiểm tra hệ thống khi số lượng request vượt mức bình thường.

Thực hiện

- Tăng số lượng request đồng thời lên mức cao
- Tập trung vào:
 - Tạo order liên tục
 - Thanh toán liên tục

Phân tích từ code

Các điểm nghẽn tiềm năng:

1. Loop trong createOrderService

```
for (const item of items) {  
  const product = await Product.findById(item.product);  
}
```

Gây nhiều query DB

2. Update nhiều collection khi thanh toán

```
await Product.findByIdAndUpdate(item.product, {  
  $inc: { soldCount: item.quantity },  
});
```

Kết quả

- Hệ thống bắt đầu chậm khi số lượng request tăng cao
- Không bị crash hoàn toàn
- Các request vẫn được xử lý (queue)

Nguyên nhân:

- MongoDB xử lý tuần tự nhiều query
- Chưa tối ưu batch query

6.4.3 Endurance Testing

Mô tả

Kiểm tra khả năng hoạt động liên tục của hệ thống trong thời gian dài.

Thực hiện

- Chạy hệ thống liên tục
- Gửi request định kỳ:
 - Tạo order
 - Lấy danh sách order
 - Thanh toán

Các yếu tố theo dõi

- Memory usage
- Response time
- Lỗi phát sinh

Kết quả

- Không xảy ra lỗi nghiêm trọng
- Không bị memory leak
- Hệ thống duy trì ổn định

6.4.4 Đánh giá tổng thể hiệu năng

Tiêu chí Đánh giá

Độ ổn định Cao

Khả năng chịu tải Trung bình

Thời gian phản hồi ~2–3s

Khả năng mở rộng Có thể cải thiện

6.4.5 Hướng tối ưu (định hướng phát triển)

Dựa trên phân tích code, có thể cải thiện:

- Tối ưu query:
 - Dùng batch query thay vì loop
- Cache menu (Redis)
- Giảm populate không cần thiết

- Queue xử lý payment

6.4.6 Kết luận

Hệ thống đáp ứng tốt yêu cầu:

- Hoạt động ổn định trong điều kiện thực tế
- Không xảy ra lỗi nghiêm trọng khi tải cao
- Có khả năng mở rộng trong tương lai

6.5 Tổng kết chương

Chương này đã trình bày quá trình kiểm thử hệ thống một cách toàn diện, bao gồm kiểm thử chức năng, khả dụng, theo dõi lỗi và hiệu năng. Kết quả kiểm thử cho thấy hệ thống đáp ứng tốt các yêu cầu đề ra, hoạt động ổn định và có khả năng triển khai trong môi trường thực tế. Đây là cơ sở quan trọng để đánh giá chất lượng và tính khả thi của sản phẩm.

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

7.1 Kết quả đạt được

Sau quá trình nghiên cứu, phân tích, thiết kế và triển khai, đề tài “Xây dựng ứng dụng quản lý cửa hàng thức ăn nhanh” đã hoàn thành các mục tiêu đề ra. Cụ thể:

- Xây dựng thành công hệ thống web hoạt động ổn định
- Triển khai chức năng gọi món thông qua mã QR giúp khách hàng đặt món nhanh chóng
- Phát triển hệ thống POS hỗ trợ nhân viên quản lý và xử lý đơn hàng hiệu quả
- Xây dựng các chức năng quản lý như: menu, bàn, đơn hàng và người dùng
- Hệ thống có thể vận hành trong môi trường thực tế ở quy mô nhỏ và vừa

7.2 Ưu điểm của hệ thống

Hệ thống đạt được một số ưu điểm nổi bật như:

- **Dễ sử dụng:** Giao diện thân thiện, thao tác đơn giản, phù hợp với nhiều đối tượng người dùng
- **Tính thực tiễn cao:** Giải quyết trực tiếp bài toán quản lý trong cửa hàng thức ăn nhanh
- **Tối ưu quy trình:** Giảm thiểu sai sót trong việc ghi nhận và xử lý đơn hàng
- **Tính linh hoạt:** Có thể mở rộng thêm các chức năng trong tương lai

7.3 Nhược điểm

Bên cạnh những kết quả đạt được, hệ thống vẫn còn một số hạn chế:

- Chưa phát triển ứng dụng mobile (native app)
- Chưa tối ưu cho hệ thống có quy mô lớn hoặc lượng truy cập cao
- Chưa tích hợp các cổng thanh toán trực tuyến
- Giao diện chưa được tối ưu hoàn toàn về trải nghiệm người dùng (UI/UX nâng cao)

7.4 Hướng phát triển trong tương lai

Trong tương lai, hệ thống có thể được cải tiến và phát triển thêm các tính năng sau:

- Phát triển ứng dụng mobile (Android/iOS) để nâng cao trải nghiệm người dùng
- Tích hợp trí tuệ nhân tạo (AI) để gợi ý món ăn dựa trên hành vi khách hàng
- Tích hợp các cổng thanh toán online (Momo, VNPAY, thẻ ngân hàng,...)
- Tối ưu hiệu năng và mở rộng hệ thống cho quy mô lớn (microservices, caching,...)
- Bổ sung chức năng báo cáo và phân tích dữ liệu kinh doanh

7.5 Tổng kết

Đề tài đã hoàn thành mục tiêu xây dựng một hệ thống quản lý cửa hàng thức ăn nhanh dựa trên nền tảng web với các chức năng cơ bản và có tính ứng dụng cao. Đây không chỉ là sản phẩm phục vụ cho việc đánh giá đồ án tốt nghiệp mà còn là nền tảng để tiếp tục phát triển thành một hệ thống hoàn chỉnh trong thực tế.

Tài Liệu tham khảo

Bộ Thông tin và Truyền thông, *Giáo trình Công nghệ phần mềm*, NXB Thông tin và Truyền thông, 2020.

Nguyễn Văn Ba, *Phân tích và thiết kế hệ thống thông tin*, NXB Đại học Quốc gia TP.HCM, 2018.

Trần Đình Quế, *Kiểm thử phần mềm*, NXB Đại học Quốc gia Hà Nội, 2019.

Phạm Hữu Khang, *Lập trình Web hiện đại với JavaScript*, NXB Khoa học và Kỹ thuật, 2021.

Nguyễn Thành Nam, *Phát triển ứng dụng Web với Node.js*, NXB Hồng Đức, 2022.

Lê Văn Phùng, *Cơ sở dữ liệu MongoDB và ứng dụng*, NXB Thống kê, 2021.

Tài liệu học tập môn Công nghệ phần mềm, Trường Đại học Công nghiệp Hà Nội.

Blog Viblo, *Xây dựng RESTful API với Node.js và Express*
<https://viblo.asia>

Trang chủ F8, *Khóa học NodeJS & MongoDB*
<https://fullstack.edu.vn>

Blog TopDev, *Kiến trúc RESTful API và ứng dụng thực tế*
<https://topdev.vn>

Mozilla, *MDN Web Docs*
<https://developer.mozilla.org>

Node.js Official Documentation
<https://nodejs.org>

Express.js Official Documentation
<https://expressjs.com>

MongoDB Documentation
<https://www.mongodb.com/docs>

OWASP, *Web Application Security Testing Guide*
<https://owasp.org>

Pressman, R. S., *Software Engineering: A Practitioner's Approach*, McGraw-Hill, 2014.

Martin Fowler, *Patterns of Enterprise Application Architecture*, Addison-Wesley, 2002.

Postman Learning Center
<https://learning.postman.com>